

Towards a Systems Foundation for Agentic Skills: Architecture, Lifecycle, and Security

Sanket Badhe
Google LLC

sanketbadhe@google.com

Deep Shah
Google LLC

shahdeep@google.com

Priyanka Tiwari
Purdue University

tiwarip@alumni.purdue.edu

Nehal Kathrotia
Google LLC

nehalk@google.com

Abstract

Autonomous large language model (LLM) agents increasingly face reliability, context consumption, and execution stability bottlenecks when deployed on complex, long-horizon tasks. While monolithic prompt engineering and stateless tool-calling paradigms struggle to scale, the field is rapidly converging toward *agentic skills*: modular procedural abstractions that externalize execution knowledge into reusable, executable, and portable artifacts. This paper establishes a unified systems foundation and reference architecture for the agentic skills ecosystem. We formalize skills as externalized procedural knowledge bridging high-level cognitive planning with deterministic execution environments, and systematically delineate the architecture across a nine-stage lifecycle: autonomous discovery, authoring and representation formats, memory storage, dynamic retrieval and routing, composition and orchestration, execution and repair, lifelong adaptation, empirical evaluation, and security governance. We further examine marketplace dynamics, public registries, and emerging adversarial threat vectors, alongside runtime verification and defense mechanisms. Finally, we categorize system implementations across software engineering, operating system navigation, embodied robotics, and scientific discovery, while highlighting critical open challenges in continual learning and benchmark realism. This work establishes agentic skills as a foundational paradigm for building scalable, robust, and verifiable autonomous language agents.

1 Introduction

1.1 Motivation: The Reliability and Reusability Bottlenecks in Language Agents

Large language model (LLM) agents, stateful systems that pursue goals over many steps by interleaving reasoning with tool calls, have become a dominant approach to automating open-ended workflows (Luo et al., 2025). Their performance degrades sharply as task horizons lengthen. Failure traces show that losses are unevenly distributed across steps: agents are disproportionately vulnerable to early-stage execution and path-resolution errors that propagate downstream and contaminate subsequent reasoning (Wang et al., 2026i; Liu et al., 2026i). Because agents rarely detect that an early step was wrong, they enter a no-recovery regime, looping or exhausting recursion limits rather than revisiting the faulty premise (Pushkin & Abbe, 2026).

A second bottleneck compounds the first: nothing accumulates. An agent that completes a task successfully gains no durable advantage the next time it encounters the same task, because procedural knowledge remains locked in transcripts discarded at session boundaries. Each episode re-derives routines the agent has already executed correctly, an observation motivating the recent wave of lifelong-learning agent frameworks (Yang et al., 2026b; Cao et al., 2026;

Hu et al., 2026b). Neither standard remedy closes the gap. Placing procedures in the context window imposes a token cost that grows with the instruction set and provides no mechanism for loading only what a task requires (Yuan et al., 2024; Gao et al., 2026). Fine-tuning writes procedural knowledge into weights, whereas the routines agents need are tied to tooling, APIs, and organizational conventions that change faster than retraining cycles accommodate (Zhuang et al., 2026).

Externalized executable skills target this second bottleneck. By packaging activation conditions, procedural instructions, tool declarations, and executable assets into modular artifacts stored outside the model, skills separate high-level planning from deterministic execution (Liu et al., 2024), persist across sessions, load only when relevant, and can be revised without retraining. This paper establishes a systems framework for how that abstraction is formalized, authored, retrieved, executed, and secured.

1.2 What Is an Agentic Skill? Formalizing the Abstraction

To establish a rigorous theoretical foundation across the literature, we formalize an externalized agentic skill as a six-tuple:

$$s = (\mathcal{A}, \mathcal{I}, \mathcal{C}, \mathcal{T}, \pi, \mathcal{E}) \quad (1)$$

where $\mathcal{A}$ denotes the activation condition, $\mathcal{I}$ represents procedural instructions or guidance, $\mathcal{C}$ denotes applicability constraints and environment preconditions, $\mathcal{T}$ represents accessible tools and interfaces, π represents the execution policy or procedural control flow, and $\mathcal{E}$ denotes the intended state transitions or post-condition effects (Yuan et al., 2024; Shen et al., 2024).

The Triad Membership Criteria: To distinguish agentic skills from adjacent computational abstractions in language agent architectures, an artifact must satisfy three core systems-level conditions:

1. **Encapsulated Modularity:** A skill is a discrete, externalized asset that bundles procedural logic, interface declarations $\mathcal{T}$, and constraints $\mathcal{C}$ into an independent package. It can be authored, versioned, validated, and updated without altering foundation model weights or modifying global agent harness code.
2. **Late-Binding Dynamic Invocation:** Unlike static runtime components that reside permanently in the context prompt, a skill is externalized and loaded on-demand. Its inclusion in active memory is governed by the dynamic evaluation of its activation function $\mathcal{A}(x_t, g) \rightarrow \{0, 1\}$ over the agent’s current state x_t and active goal g .
3. **Procedural State Transformation:** A skill encodes reusable operational knowledge, dictating how to execute a task and handle potential errors through policy π , rather than recording declarative facts or logging chronological histories.

Disambiguating Borderline Cases: Applying these criteria resolves common ambiguities at the boundaries between skills and related concepts:

- **Prompts vs. Skills:** A monolithic system prompt injected globally at session initialization, such as a general directive to format all responses in structured JSON, provides static agent parameterization; because it lacks modular encapsulation and late-binding retrieval, it is not a skill. Conversely, a task-specific standard operating procedure for refactoring React components, stored in a procedural library and injected into the prompt only when frontend source code is encountered, satisfies all three criteria and represents an Instructional Skill (Ling et al., 2026).
- **Atomic Tools vs. Skills:** A standard API schema provided directly to the model, such as a basic web search endpoint, is a stateless functional primitive $\mathcal{T}$ functioning simply as an atomic tool. In contrast, an execution wrapper that bundles this API with multi-step orchestration, state tracking, and recovery logic (such as inspecting HTTP status codes, falling back to cached archive endpoints upon request timeouts, and parsing DOM trees) autonomously manages procedural control flow, elevating it to a Tool-Execution Skill (Yuan et al., 2024; Chen et al., 2026c).

- **Workflows and Frameworks vs. Skills:** The primary cognitive architecture of an agent harness, such as a hardcoded ReAct or Plan-and-Solve control loop, constitutes runtime infrastructure. However, when a composite Directed Acyclic Graph of sub-actions (such as compiling code, running unit test suites, parsing execution tracebacks, and formatting patch diffs) is serialized as an external asset, stored in a repository, and retrieved dynamically to solve a sub-goal, it functions as a modular Composite Skill (Shen et al., 2024; Xia et al., 2026c).
- **Episodic Memory and Plans vs. Skills:** Episodic memory acts as a passive historical log of past trajectory observations $\mathcal{E}$ without active execution logic, whereas transient plans represent ephemeral, single-use scratch-pads generated for the current context. Skills, by contrast, are persistent, generalized, and reusable procedural templates that endure across multi-session lifetimes (Packer et al., 2023; Summers et al., 2024).

A systematic comparison of agentic skills against these alternate computational abstractions is summarized in Table 1. As shown, skills represent the unique abstraction that unites persistence across resets, modular external execution, runtime composability, and non-parametric modifiability.

Table 1: **Comparison of Computational Abstractions (§1.2).** Skills are the only abstraction that combines persistence, external execution, runtime composition, and modification without retraining; however, unlike workflows and typed tools, they cannot be fully verified before invocation, which is the source of the security properties analyzed in §3.9.

Abstraction	Components Instantiated	Persists across resets	Invoked by	Executes outside	Composable at runtime	Modifiable w/o retrain	Verifiable before use	Cost per invocation	Reference
System prompt	$\mathcal{I}$	×	always-on	×	×	✓	×	full, every turn	Yao et al. (2023b); Wei et al. (2022); Summers et al. (2024)
Episodic memory	$\mathcal{E}$ (traces only)	✓	similarity match	×	×	✓	×	retrieval only	Packer et al. (2023); Park et al. (2023); Zhou et al. (2026)
Workflow / DAG	$\mathcal{I}, \pi$ ($\mathcal{A}$ fixed at authoring)	✓	scheduler	partial	×	×	✓	per node	Hong et al. (2024); Wu et al. (2024); Qian et al. (2024)
Tool / API	$\mathcal{T}, \mathcal{E}$	✓	model emits call	✓	✓	×	✓	per call	Schick et al. (2023); Patil et al. (2024); Yuan et al. (2024)
Agentic skill	$\mathcal{A}, \mathcal{I}, \mathcal{C}, \mathcal{T}, \pi, \mathcal{E}$	✓	conditional on state	✓	✓	✓	partial	metadata, then body	Wang et al. (2024); Yang et al. (2026b); Lu et al. (2026b); Ling et al. (2026)

1.3 Origins of Skill-Based Agent Design

The development of agentic skills builds upon a technical evolution in model augmentation. The first phase involved static prompting paradigms like ReAct, SayCan, and Inner Monologue, demonstrating that models could perform multi-step reasoning Summers et al. (2024). The second phase introduced atomic tool-calling frameworks such as Toolformer, Gorilla, MRKL, APiBank, HuggingGPT, and ViperGPT, prompting language models to emit tokens representing single API calls Schick et al. (2023); Patil et al. (2024). The third phase transitioned to programmatic loops and persistent state-tracking, as seen in Voyager, MemGPT, Generative Agents, Reflexion, and Self-Refine Wang et al. (2024); Packer et al. (2023). Finally, the modern fourth phase establishes packaged agent skills as a highly capable abstraction for platforms like SWE-Agent, MetaGPT, AutoGPT, WebArena, and Mind2Web, enabling the secure deployment of pre-compiled directories Shen et al. (2024).

1.4 Summary of Contributions

This paper provides a unified systems-level foundation for the emerging landscape of agentic skills, formalizing a rapidly growing paradigm into an end-to-end, lifecycle-oriented architecture. In summary, our core contributions are:

1. **Formal Foundations and Mathematical Abstraction:** We formalize agentic skills under a unified six-tuple abstraction $s = (\mathcal{A}, \mathcal{I}, \mathcal{C}, \mathcal{T}, \pi, \mathcal{E})$, establishing a clear conceptual boundary that separates externalized procedural knowledge from static prompting, episodic memories, and stateless API calls (Table 1).

2. **Procedural Skill Reference Architecture:** We synthesize the procedural lifecycle across nine core phases (Discovery, Representation, Memory Storage, Dynamic Routing, Orchestration, Execution/Repair, Adaptation, Evaluation, and Security Governance), supported by ten comprehensive architectural comparison tables.
3. **Taxonomy of Capabilities and Domains:** We classify skill implementations across functional categories (Instructional SOPs, Tool Calling, Latent Reasoning, Meta-Skills) and four operational environments: Software Engineering, GUI/OS Navigation, Embodied Robotics, and Scientific Discovery (Table 9).
4. **Ecosystems, Marketplaces, and Threat Modeling:** We analyze real-world marketplace dynamics, public skill registries, empirical execution suites, and an adversarial threat taxonomy with corresponding runtime defenses.

2 Foundations of Skill-Based Agents

2.1 Cognitive Architectures and Procedural Memory

In cognitive psychology, memory is broadly divided into declarative memory, which handles semantic facts, and procedural memory, which governs the procedural mechanics of task execution Sumers et al. (2024). Computational cognitive architectures model human problem-solving as a tight, iterative loop where declarative knowledge is compiled into procedural rules over time Sumers et al. (2024). Within the context of language agents, agentic skills serve as the non-parametric procedural memory of the autonomous system. While parametric model weights represent a static semantic memory, and vector databases represent a transient episodic memory, skills provide the modular execution substrate required for repeatable tasks Packer et al. (2023). When faced with a familiar problem, the agent retrieves the corresponding procedural skill and executes it directly, bypassing slow reasoning steps Sumers et al. (2024).

2.2 Skills as Externalized Procedural Knowledge

The core architectural innovation lies in the externalization of this procedural memory. Attempting to feed extensive procedural instructions directly into the context window inflates inference latency and degrades accuracy Chacko et al. (2026). Externalizing procedural knowledge into packaged files resolves these limitations. Furthermore, externalization enables the system-level principle of progressive disclosure, where the agent reads only the lightweight metadata first, loading full implementation code only during active execution Yuan et al. (2024). Because these skills are stored as plain-text assets, they can be updated without requiring expensive model retraining loops Zhuang et al. (2026).

We formalize the core primitives of this framework under a unified mathematical abstraction:

Definition 1 (Skill): A skill s is a reusable procedural abstraction consisting of activation conditions ($\mathcal{A}$), procedural instructions ($\mathcal{I}$), execution constraints ($\mathcal{C}$), tool interfaces ($\mathcal{T}$), execution policy (π), and intended effects ($\mathcal{E}$) Yuan et al. (2024):

$$s = (\mathcal{A}, \mathcal{I}, \mathcal{C}, \mathcal{T}, \pi, \mathcal{E}) \quad (2)$$

Definition 2 (Agent State): The agent state at time t is denoted as $x_t \in \mathcal{X}$, capturing conversation history, internal memory, environment observations, tool outputs, active goals $g \in \mathcal{G}$, and retrieved documents Packer et al. (2023).

Definition 3 (Skill Activation): A skill activates conditionally based on the agent state x_t and the active task goal g :

$$\mathcal{A}(x_t, g) \rightarrow \{0, 1\} \quad (3)$$

where $\mathcal{A}(x_t, g) = 1$ indicates that the skill should trigger, formalizing dynamic routing, contextual applicability, and planner-guided invocation Zheng et al. (2026). Crucially, to prevent unauthorized execution and privilege escalation, activation is governed by a capability-based permission constraint:

$$\mathcal{A}(x_t, g) = 1 \Rightarrow \mathcal{T}_s \subseteq \mathcal{T}_{\text{permitted}}(x_t) \quad (4)$$

A skill may fire only if its declared tool set $\mathcal{T}_s$ is a strict subset of the tools permitted by the active execution context $\mathcal{T}_{\text{permitted}}(x_t)$ Li et al. (2026g). This containment formalizes the capability-based permission model required for secure

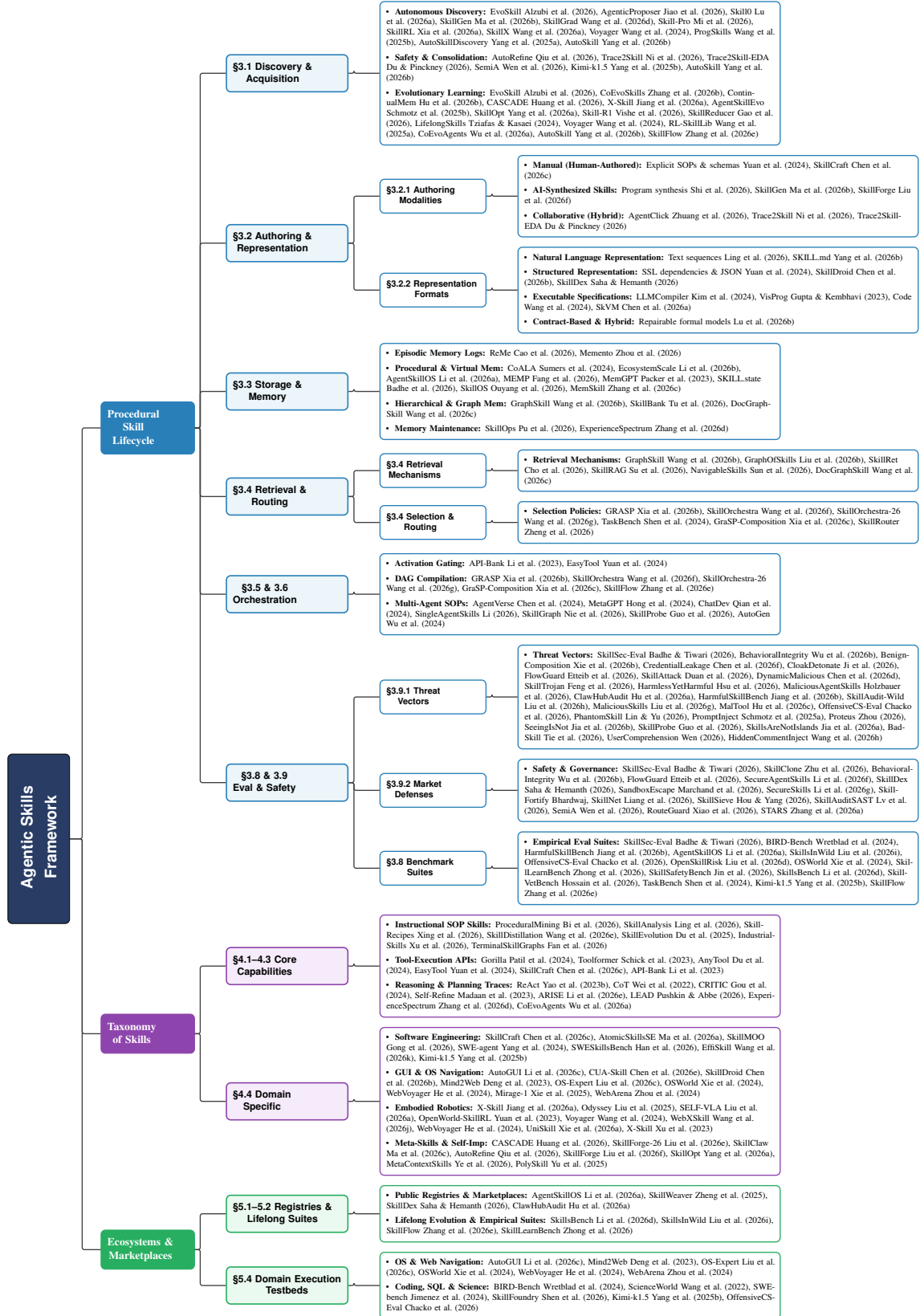


Figure 1: Architectural Blueprint and Systems Landscape of the Agentic Skills Framework, categorizing key system methodologies, lifecycle stages, and domain environments.

agent execution, defining privilege escalation as any violation of this subset condition ($\mathcal{T}_s \not\subseteq \mathcal{T}_{\text{permitted}}(x_t)$), and establishing a formal boundary between runtime sandboxing and pre-deployment static auditing.

Definition 4 (Skill Execution): When activated, a skill induces a conditional state transition:

$$x_{t+1} = s(x_t) = \pi(x_t, \mathcal{I}, \mathcal{T}) \quad \text{if} \quad \mathcal{A}(x_t, g) = 1 \quad (5)$$

This formalizes procedural execution, deterministic tool interaction, and reasoning under conditional invocation Zheng et al. (2026).

3 Procedural Skill Lifecycle in LLM Agents

The procedural skill lifecycle represents the core operational architecture of modern agent platforms, governing how skills transition from initial creation to runtime deployment, adaptation, and eventual decommissioning. Managing these stages dynamically is critical to maintaining a reliable, safe, and efficient agent capability ecosystem Yang et al. (2026b). This section details each stage of the lifecycle, unifying them under a consistent mathematical framework. While this section details the operational lifecycle of skills, a comprehensive functional classification across domains is provided in Section 4 (Table 9), and an analysis of skill ecosystems and marketplaces is provided in Section 5 (Table 10).

3.1 Skill Discovery and Acquisition

When deployed in open-ended or unfamiliar operational environments, autonomous language agents cannot rely exclusively on fixed, pre-compiled skill libraries. To achieve genuine autonomy, agents require mechanisms to autonomously discover, formalize, and consolidate new procedural capabilities from ongoing interactive experience (Yang et al., 2026b; Zhang et al., 2026b). As summarized in Table 2, contemporary frameworks approach discovery from fundamentally different assumptions regarding trajectory sources, failure signal utilization, and pre-admission verification rigor.

3.1.1 Formalizing the Discovery and Acquisition Pipeline

We formalize skill discovery as the mathematical problem of extracting reusable sub-policies from a distribution of unstructured, long-horizon rollout logs $\mathcal{H} = \{\tau^{(1)}, \tau^{(2)}, \dots, \tau^{(N)}\}$, where each trajectory $\tau = (x_1, a_1, r_1, \dots, x_H)$ records sequences of environment observations, actions, intermediate rewards, and conversational context (Zhang et al., 2026b; Yang et al., 2025a). The discovery operator $\mathbb{D}$ maps the trajectory history $\mathcal{H}$ to a candidate set of unverified skill representations $\mathcal{S}_{\text{new}}$:

$$\mathcal{S}_{\text{new}} = \mathbb{D}(\mathcal{H}) \quad (6)$$

In practice, $\mathbb{D}$ operates by identifying an optimal sub-trajectory $\tau_{i:j} \subseteq \tau$ (with $1 \leq i < j \leq H$) that satisfies two conditions: (1) *persistent environmental outcome*, inducing a predictable state transformation $\mathcal{E}$, and (2) *entropy minimization*, reducing the branching factor of future policy decisions (Xia et al., 2026a; Shu et al., 2017). Once a sub-trajectory is partitioned, the agent induces an applicability context $\mathcal{C} \subseteq \mathcal{X}$ defining the region of state space where the procedural routine remains numerically or semantically stable, which defines the initial triggering condition $\mathcal{A}_{\text{new}}$:

$$\mathcal{A}_{\text{new}}(x_t, g) = 1 \iff x_t \in \mathcal{C} \quad (7)$$

Following discovery, the *Skill Acquisition* operator $\mathbb{A}$ refines and consolidates raw candidates $s \in \mathcal{S}_{\text{new}}$ into production-ready skills $s^* = (\mathcal{A}^*, \mathcal{I}^*, \mathcal{C}^*, \mathcal{T}^*, \pi^*, \mathcal{E}^*)$ committed to persistent procedural memory $\mathcal{M}$:

$$\mathcal{M}_{t+1} = \mathcal{M}_t \cup \{s^*\} \quad (8)$$

The acquisition pipeline optimizes the instructions $\mathcal{I}^*$ and executable policy π^* to maximize expected operational reward under applicability constraints, subject to a pre-admission verification constraint $V(s^*) = 1$:

$$s^* = \arg \max_s \mathbb{E}_{x \sim \mathcal{C}} [R(s, x)] \quad \text{subject to} \quad V(s^*) = 1 \quad (9)$$

where $V(s^*) \in \{0, 1\}$ formalizes pre-commit verification (e.g., unit test pass rates, static AST analysis, or formal Datalog invariant proving) to guarantee that newly consolidated skills do not introduce syntactic regressions, interface drift, or privilege escalation vulnerabilities (Wen et al., 2026; Xia et al., 2025).

Table 2: **Discovery, Acquisition & Authoring (§3.1 & §3.2.1).** Comparison of procedural skill creation and evolution frameworks ($n = 19$ systems). Sorted primarily by **Authoring modality** (*Autonomous RL* $\rightarrow$ *AI-synthesized* $\rightarrow$ *Hybrid* $\rightarrow$ *Manual*), then secondarily by **Updates over time**. **Columns:** **System** (procedural creation/evolution framework); **Core Contribution** (primary algorithmic or structural innovation); **Trajectory Source** (input origin of procedural knowledge: dialogue, task rollout, embodied, or manual); **Authoring Modality** (abstraction generation mechanism: autonomous RL, AI-synthesized, hybrid HITL, or manual); **Failure Signal Usage** (how failure traces inform synthesis: repair signal, contrastive RL pairs, rejection filter, or not used); **Verified Before Commit** (pre-admission quality gating: formal, unit test, LLM judge, HITL review, or none); **Updates Over Time** (continual adaptation target: instructions, policy, library membership, or none).

System	Core Contribution	Trajectory Source	Authoring Modality	Failure Signal Usage	Verified Before Commit	Updates Over Time
<i>Autonomous Reinforcement Learning (RL-Induced Procedural Abstractions)</i>						
SkillRL Xia et al. (2026a)	RL-induced procedural policy	Task rollout	Autonomous RL	Contrastive pairs	Unit test	Policy, membership
Skill-Pro Mi et al. (2026)	Non-parametric PPO over Skill-MDP	Task rollout	Autonomous RL	Contrastive pairs	PPO Gate	Membership, instructions
Skill-R1 Vishe et al. (2026)	RL-based skill evolution	Task rollout	Autonomous RL	Contrastive pairs	Unit test	Membership
Skill0 Lu et al. (2026a)	In-context RL internalization	Task rollout	Autonomous RL	Contrastive pairs	None	Policy (internalized)
<i>AI-Synthesized Skills (LLM-Generated SOPs, Trajectory Consolidation & Textual Evolution)</i>						
SkillOpt Yang et al. (2026a)	Controllable text-space optimizer	Task rollout	AI-synthesized	Rejection filter	Unit test, LLM judge	Instructions
Voyager Wang et al. (2024)	Continual embodied skill library	Embodied	AI-synthesized	Repair signal	Unit test	Membership
AutoSkill Yang et al. (2026b)	Experience-driven lifelong dialogue	Dialogue, rollout	AI-synthesized	Not used	LLM judge	Membership
EvoSkill Alzubi et al. (2026)	Automated evolutionary discovery	Task rollout	AI-synthesized	Repair signal	LLM judge	Instructions, membership
ProgSkills Wang et al. (2025b)	Programmatic skill induction	Task rollout	AI-synthesized	Not used	Unit test	Membership
SkillGen Ma et al. (2026b)	Inference-time verified synthesis	Task rollout	AI-synthesized	Contrastive pairs	Unit test, HITL	Membership
SkillX Wang et al. (2026a)	Automated skill construction	Task rollout	AI-synthesized	Not used	Unit test	Membership
Trace2Skill Ni et al. (2026)	Distills trajectory failure lessons	Task rollout	AI-synthesized	Repair signal	Unit test	Membership
Trace2Skill-EDA Du & Pinckney (2026)	Trajectory exploratory analysis	Task rollout	AI-synthesized	Repair signal	Unit test	Membership
AutoRefine Qiu et al. (2026)	Refining trajectory expertise	Task rollout	AI-synthesized	Repair signal	LLM judge	Membership
CASCADE Huang et al. (2026)	Cumulative skill development	Task rollout	AI-synthesized	Repair signal	Unit test	Membership
CoEvoSkills Zhang et al. (2026b)	Co-evolving decision & skill banks	Task rollout	AI-synthesized	Repair signal	LLM judge	Membership, instructions
PSN Shi et al. (2026)	Programmatic skill networks	Embodied	AI-synthesized	Repair signal	Unit test	Membership
<i>Hybrid & Manual Authoring (Human-in-the-Loop Review & Hand-Authored SOP Wrappers)</i>						
AgentClick Zhuang et al. (2026)	Skill-based HITL review	Dialogue	Hybrid	n/a	HITL review	Instructions
EasyTool Yuan et al. (2024)	Manual SOP schema wrappers	None (manual)	Manual	n/a	None	None

Notes · Column Value Definitions. **Trajectory Source:** Origin of procedural experience (*Task rollout*: agent-environment interaction traces; *Dialogue*: multi-turn conversational logs; *Embodied*: physical/3D simulator actions; *None (manual)*: human-authored from scratch). **Authoring Modality:** Creation mechanism (*Autonomous RL*: reinforcement learning optimization; *AI-synthesized*: LLM-generated code/SOPs from trajectories; *Hybrid*: AI synthesis with Human-in-the-Loop review; *Manual*: human engineering). **Failure Signal Usage:** How failed rollouts inform synthesis (*Contrastive pairs*: success/failure trajectories used as RL reward signals; *Repair signal*: error tracebacks fed back to LLM for iterative correction; *Rejection filter*: failed rollouts discarded without learning; *Not used*: only successful trajectories retained). **Verified Before Commit:** Pre-admission quality gating (*Unit test*: executable validation against test suites; *LLM judge*: model-based utility/schema inspection; *HITL review*: human review before commit; *None*: unverified admission). **Updates Over Time:** Target of continual adaptation (*Instructions*: textual SOP/prompt refinement; *Policy*: parametric weight or RL policy updates; *Membership*: adding/pruning skills in the library; *None*: static library).

3.1.2 Comparative Paradigms: Autonomous RL vs. Inductive AI Synthesis

The literature diverges sharply on how the discovery operator $\mathbb{D}$ and acquisition operator $\mathbb{A}$ are instantiated:

Autonomous Reinforcement Learning (Option Discovery): Frameworks such as SkillRL (Xia et al., 2026a), Skill-Pro (Mi et al., 2026), Skill-R1 (Vishe et al., 2026), and Skill0 (Lu et al., 2026a) ground skill discovery in classical hierarchical reinforcement learning and Markov Decision Process (MDP) option discovery. These architectures discover sub-policies by maximizing intrinsic reward signals (e.g., state reachability, bottleneck state visitation, or contrastive advantage) over task rollouts. For instance, SkillRL (Xia et al., 2026a) discovers atomic execution options by training policy sub-networks against environment rewards, distilling rollouts through a teacher model into a two-tier SKILLBANK (universal general skills S_g and task-specific skills S_k), before co-evolving the library with policy updates via Group Relative Policy Optimization (GRPO). Similarly, Skill-Pro (Mi et al., 2026) formalizes a Skill-MDP where non-parametric Proximal Policy Optimization extracts natural-language semantic gradients ($g_i = \nabla_{\text{sem}}(\tau_i, \omega)$) from trajectory batches, gating candidate skill admissions through a clipped surrogate verification functional (PPO Gate) and maintaining pool efficiency via online advantage-score pruning. *Underlying Assumption & Limitation:* Autonomous RL methods assume environment rewards are dense and execution environments are computationally cheap to simulate. While they achieve high execution fidelity in bounded game environments or robotics simulations (Wu et al., 2025; Liu et al., 2026a), their sample complexity (10^4 – 10^6 rollout steps) becomes prohibitively expensive when applied to real-world software engineering repositories or API-constrained web agents where every rollout consumes external model tokens.

Inductive AI Synthesis (Code and SOP Induction): To overcome the sample inefficiency of RL, architectures such as Voyager (Wang et al., 2024), ProgSkills (Wang et al., 2025b), SkillGen (Ma et al., 2026b), SkillX (Wang et al., 2026a), and AutoSkill (Yang et al., 2026b) leverage the in-context reasoning of frontier LLMs to synthesize skill code and markdown SOPs from as few as 1–5 demonstration traces. In Voyager (Wang et al., 2024), an iterative prompting mechanism converts interactive Minecraft environment feedback and execution errors into reusable JavaScript control programs stored in a vector database. In dialogue-centric and workflow domains, AutoSkill (Yang et al., 2026b) continually parses multi-turn conversation trajectories, distilling conversational insights into structured SKILL.md procedural files without human supervision. *Underlying Assumption & Limitation:* Inductive synthesis assumes the underlying foundation model possesses sufficient latent programming capabilities to accurately generalize from noisy traces. In practice, unsupervised synthesis without execution grounding frequently produces brittle skills that overfit to idiosyncrasies of the specific rollout trace, suffering from parameter hallucinations and ambiguous activation boundaries.

Evolutionary Search in Text Space: Bridging RL and inductive synthesis, EvoSkill (Alzubi et al., 2026), SkillOpt (Yang et al., 2026a), and CASCADE (Huang et al., 2026) frame skill discovery as evolutionary optimization directly over textual prompt and code spaces. EvoSkill applies genetic operators (mutation, crossover, and fitness selection) over candidate skill instructions, using task pass rates and execution latencies as multi-objective fitness scores. SkillOpt introduces a controllable text-space optimizer that refines instruction formulations using rejection filtering over task rollouts.

3.1.3 The Role of Failure Signals: Positive-Only vs. Contrastive Mining

A major point of divergence across discovery architectures is whether and how execution failure traces are utilized:

- **Positive-Only Induction:** Early architectures, including Voyager (Wang et al., 2024), ProgSkills (Wang et al., 2025b), and AutoSkill (Yang et al., 2026b), discard failed trajectories entirely, inducing skills strictly from successful rollout trajectories. While computationally straightforward, positive-only induction leaves skills blind to operational failure modes, producing procedures that fail catastrophically when encountering runtime exceptions.
- **Contrastive Mining and Error Tracebacks:** Recent systems, such as Trace2Skill (Ni et al., 2026), Trace2Skill-EDA (Du & Pinckney, 2026), AutoRefine (Qiu et al., 2026), CoEvoSkills (Zhang et al., 2026b), and SkillGen (Ma et al., 2026b), demonstrate that analyzing paired success-failure traces significantly improves skill robustness. Trace2Skill (Ni et al., 2026) deploys asymmetric analyst sub-agents: a success analyst A^+ extracts positive execution patterns while an interactive error analyst A^- inspects execution tracebacks,

file diffs, and ground-truth mismatches to generate targeted patch proposals. These patches are merged through hierarchical many-to-one consolidation into standard operating procedures (e.g., formula recalculation, write-back verification), drastically improving wall-clock consolidation efficiency over sequential prompt editing. In hardware engineering, Trace2Skill-EDA (Du & Pinckney, 2026) proves that coupling dense verifier feedback with task-wise error distillation substantially boosts complex EDA script execution success. SkillGen (Ma et al., 2026b) clusters failure and success summaries via k -means embeddings to synthesize contrastive pre-condition boundaries $\mathcal{C}$, preventing the skill from triggering in out-of-distribution states.

3.1.4 Pre-Admission Quality Gating and Verification

Before a discovered skill candidate is committed to persistent memory $\mathcal{M}$, frameworks enforce varying degrees of quality gating $V(s)$:

- **Unit Testing and Sandbox Execution:** Systems like Voyager (Wang et al., 2024), SkillIRL (Xia et al., 2026a), ProgSkills (Wang et al., 2025b), and PSN (Shi et al., 2026) demand that synthesized skills pass deterministically constructed unit tests inside isolated sandbox containers prior to library admission.
- **LLM-as-a-Judge Vetting:** Frameworks operating on natural language SOPs (AutoRefine (Qiu et al., 2026), CASCADE (Huang et al., 2026), EvoSkill (Alzubi et al., 2026)) deploy secondary LLM judges to inspect candidate skill descriptions, evaluating clarity, modularity, and redundancy against the existing skill bank.
- **Formal Contract Proving:** Verification frameworks like SemiA (Wen et al., 2026) translate skill instructions and execution bodies into formal Datalog rules, checking temporal safety invariants before permitting deployment to production registries.
- **Human-in-the-Loop Review:** Collaborative platforms like AgentClick (Zhuang et al., 2026) inject human verification checkpoints where human engineers approve or modify synthesized procedures before deployment into production workflows.

3.1.5 Critical Methodological Weaknesses and Open Bottlenecks

Our synthesis identifies three fundamental bottlenecks currently limiting autonomous discovery:

1. **The Exploration-Exploitation Gap in Sparse Environments:** In complex software engineering repositories (e.g., SWE-bench (Jimenez et al., 2024)) or open-world operating systems (OSWorld (Xie et al., 2024)), unguided exploration rarely encounters successful end-to-end trajectories by chance. Without human demonstration traces or heavily shaped curricula, autonomous discovery algorithms fail to bridge the initial zero-to-one capability gap.
2. **Skill Proliferation and Memory Dilution:** Unchecked autonomous discovery rapidly floods the skill repository with low-utility, overspecialized, or near-duplicate skill files ($n > 10^3$). As demonstrated by Fang et al. (2026) and Liu et al. (2026i), large uncured skill pools saturate retrieval indices, increasing semantic confusability and severely degrading downstream task completion rates.
3. **The LLM Judge Reliability Gap:** Relying on LLM judges for pre-admission gating $V(s)$ introduces silent failure modes. As shown in empirical vetting studies (Wu et al., 2026b; Hossain et al., 2026), LLM referees suffer from sycophancy and frequently overlook subtle syntax errors and stealthy payload injections in multi-file dependencies, underscoring the urgent need for hybrid formal-neural verifiers (Wen et al., 2026).

3.2 Skill Authoring and Representation

Once a skill is discovered or engineered, formalizing, serializing, and packaging its procedural assets for heterogeneous execution harnesses represents a core architectural challenge. Skill authoring dictates how procedural expertise is initially constructed, while representation format determines the trade-off between runtime computational expressivity, harness portability, and pre-execution safety verifiability (Ling et al., 2026; Chen et al., 2026a).

3.2.1 Skill Authoring Modalities

Agentic frameworks construct procedural assets through three primary authoring paradigms:

- **Manual Human Engineering:** Software engineers and domain experts explicitly author standard operating procedures (SOPs), declarative parameter schemas, and native scripts (Yuan et al., 2024). Frameworks like EasyTool (Yuan et al., 2024) demonstrate that hand-crafted, standardized tool wrappers eliminate parameter hallucinations and compress verbose REST documentation. While manual authoring guarantees deterministic predictability, domain correctness, and strict adherence to organizational policy, it presents an intractable human engineering bottleneck that cannot scale horizontally to thousands of dynamic, evolving tools.
- **Autonomous AI Synthesis:** The agent autonomously synthesizes, evaluates, and registers its own code assets and execution heuristics without human intervention (Yang et al., 2026b; Jiao et al., 2026; Wang et al., 2025b). Architectures such as Programmatic Skill Networks (PSN) (Shi et al., 2026) and SkillGen (Ma et al., 2026b) leverage verifier-guided code generation and contrastive trajectory induction to write executable Python/JavaScript functions and markdown guidelines directly from rollout experience. While scalable, purely autonomous synthesis is susceptible to generating fragile preconditions and unverified edge-case workarounds if not grounded by rigorous sandboxed execution checks.
- **Collaborative Hybrid Authoring (Human-in-the-Loop):** Bridging manual precision with autonomous scale, hybrid authoring couples AI generation with interactive human oversight (Ni et al., 2026). In systems like AgentClick (Zhuang et al., 2026), human operators define high-level capability schemas, safety constraints, and permission boundaries, while the foundation model generates the underlying procedural implementation and verifies it through sandboxed unit testing, inserting human approval checkpoints before terminal execution.

3.2.2 Skill Representation Formats and Verifiability Trade-Offs

A comprehensive architectural comparison of the five primary representation format classes is provided in Table 3.

To accommodate diverse deployment runtime constraints, the literature structures skills across five representation paradigms:

1. Natural Language Representations (SKILL.md / SOPs): Natural language representation encodes procedural knowledge as human-readable markdown files (SKILL.md) containing task descriptions, step-by-step guidelines, and execution checklists (Yang et al., 2026b; Ling et al., 2026). This format, standardized by community package managers like SkillDex (Saha & Hemanth, 2026) and audited by SkillAnalysis (Ling et al., 2026), maximizes portability across arbitrary LLM harnesses, as instructions $\mathcal{I}$ can be injected directly into prompt contexts without specialized interpreters. However, natural language representations suffer from severe verbosity (imposing high load-time token costs), semantic ambiguity in activation conditions $\mathcal{A}$, and zero pre-execution verifiability, forcing agents to rely purely on unconstrained prompt-following (Yuan et al., 2024).

2. Structured Representations (JSON / DSL / State Machines): To resolve natural language verbosity and enforce strict parameter schemas, structured formats represent skills via JSON schemas, YAML frontmatter, or Domain-Specific Languages (DSLs) (Yuan et al., 2024; Schick et al., 2023). In GUI and mobile automation, SkillDroid (Chen et al., 2026b) compiles complex Android UI interactions into finite state transition graphs $G = (V, E)$, where vertices represent UI trees and edges denote validated tap/scroll actions. Similarly, Toolformer (Schick et al., 2023) encodes tools as structured API calling tokens $[\text{API}(\dots) \rightarrow \text{res}]$. Structured formats enable pre-execution parameter typing and schema validation ($\mathcal{T}$), but lack arbitrary procedural computation capabilities (π), restricting the agent to declarative parameter passing.

3. Executable Programmatic Specifications: Going beyond declarative metadata, executable specifications serialize skills as native programmatic code (Python scripts, JavaScript modules) (Wang et al., 2024; 2025b; Kim et al., 2024). In Voyager (Wang et al., 2024) and ProgSkills (Wang et al., 2025b), skills are executable functions encapsulating complex algorithmic control flow, including state queries, loops, and conditional branching. To execute these routines across heterogeneous models, virtual machines like SkVM (Chen et al., 2026a) decompose 118,000 skills into primitive capability profiles with dynamic environment binding. Executable specifications provide full computational expressivity

Table 3: **Representation Formats & Security Trade-Offs (§3.2.2).** Comparison of procedural skill representation format classes (5 format categories, not individual papers). Rows 1–4 are sorted monotonically by pre-execution verifiability (*none* $\rightarrow$ *schema* $\rightarrow$ *static-AST* $\rightarrow$ *formal*); Row 5 (*Hybrid & adaptive*) is placed by its lower bound and sits intentionally off the main gradient. Demonstrates that verifiability and portability trade off monotonically across rows 1–4. Injection channel tracks neither; it is determined by execution capability, which varies independently of verifiability.

Format Class	Pre-Execution Verifiability	Portability	Execution Capability	Injection Channel	Load-Time Token Cost	Repair Signal on Failure	Representative Exemplars
Natural language	none (no component)	high	$\times$ none	prose	high	re-prompt only	SKILL.md (AutoSkill) Yang et al. (2026b), SkillDex Saha & Hemanth (2026), SkillAnalysis Ling et al. (2026), PromptSkills Maloyan & Namiot (2026), ProceduralMining Bi et al. (2026)
Structured (JSON / $\mathcal{T}$ (schema) DSL)		high	$\times$ none	schema strings	medium	schema error	EasyTool Yuan et al. (2024), SkillDroid Chen et al. (2026b), Toolformer Schick et al. (2023)
Executable specification	π (static analysis)	medium	$\checkmark$ arbitrary code	code + prose	high	stack trace	Voyager Wang et al. (2024), LLMCompiler Kim et al. (2024), VisProg Gupta & Kembhavi (2023), SkVM Chen et al. (2026a), ProgSkills Wang et al. (2025b)
Contract-based	$\pi, \mathcal{E}$ (pre/postconditions)	low	$\checkmark$ constrained	code + prose	medium-high	counterexample	ContractSkill Lu et al. (2026b), SemiA Wen et al. (2026), SkillAuditSAST Lv et al. (2026)
Hybrid & adaptive	schema + π (static analysis)	medium	$\checkmark$ arbitrary code	code + prose	low at load, high on activation	schema error $\rightarrow$ stack trace	GraphSkill Wang et al. (2026b), PSN Shi et al. (2026)

Notes · Column Value Definitions. **Pre-Execution Verifiability:** Pre-loading safety/correctness checking (*none*: uncheckable raw prompt instructions; *schema*: JSON schema or OpenAPI declarative parameter checking; *static-AST*: code syntax checking, static AST linting, and type inference; *formal*: mathematical pre/postcondition checking and Datalog/SMT solver verification; *schema* + *static-AST*: mixed header schema validation and code body AST linting). **Portability across harnesses:** Framework compatibility (*high*: runs unmodified on arbitrary LLM prompts/agents; *medium*: requires standard code sandbox or Python runtime; *low*: requires specialized formal verification engine or solver). **Execution Capability:** Runtime computational power ($\times$ *none*: declarative text or schema only, cannot execute code; $\checkmark$ *arbitrary code*: executes general-purpose programming language statements; $\checkmark$ *constrained*: executes bounded or repairable formal contracts/Datalog rules). **Injection Channel:** Where malicious payloads enter (*prose*: natural language instructions/descriptions; *schema strings*: restricted to string parameter values and argument descriptions; *code* + *prose*: executable function bodies, imports, or docstrings). **Load-Time Token Cost:** Context window overhead (*high*: raw unindexed prompt instructions or full function bodies; *medium*: schema signatures; *medium-high*: formal invariants + code; *low at load, high on activation*: progressive disclosure loading metadata first). **Repair Signal on Failure:** Actionable artifact returned on execution failure (*re-prompt only*: unstructured textual reflection; *schema error*: declarative parameter mismatch; *stack trace*: runtime execution traceback; *counterexample*: formal invariant violation trace; *schema error* $\rightarrow$ *stack trace*: progressive multi-layer repair).

(π), deterministic error handling via runtime stack traces, and static Abstract Syntax Tree (AST) linting, but require sandboxed execution environments and introduce significant code execution security vulnerabilities.

4. Contract-Based and Formal Specifications: To guarantee behavioral safety in high-stakes domains, contract-based representations wrap procedural execution in verifiable pre-conditions, post-conditions, and invariant contracts (Lu et al., 2026b; Wen et al., 2026). ContractSkill (Lu et al., 2026b) introduces repairable contracts for multimodal web agents, generating structured counterexamples upon contract violation to guide localized error repair. In static auditing, SemiA (Wen et al., 2026) lifts hybrid prose and code into Datalog fact bases within the Skill Description Language (SDL), enabling automated SMT and logic solvers to mathematically prove the absence of data leakage and privilege escalations prior to execution. While providing the highest safety guarantees, contract-based formats suffer from low portability, requiring specialized formal verification solvers.

5. Hybrid and Adaptive Representations: Reconciling cognitive flexibility with systems-level safety, hybrid formats package natural language guidelines, declarative schemas, and executable scripts into structured hierarchical directories (Shi et al., 2026; Wang et al., 2026b). In GraphSkill (Wang et al., 2026b), structured documentation graphs guide hierarchical retrieval, formalizing dependencies as a graph $G_s = (V_s, E_s)$ where vertices V_s represent procedural execution steps and directed edges E_s encode prerequisite data flows. In memory theory, ExperienceSpectrum (Zhang et al., 2026d) unifies raw traces (1:1), episodic summaries, procedural skills (10–100 $\times$), and symbolic rules (1000 $\times$) along a single continuous compression continuum. To support vector search across massive registries, encoder models f_θ map skills into continuous vector embeddings:

$$e_s = f_\theta(\mathcal{A}, \mathcal{I}, \mathcal{C}) \in \mathbb{R}^d \quad (10)$$

enabling fast similarity routing while supporting progressive disclosure, loading lightweight metadata e_s at search time and pulling executable code π only upon conditional activation (Cho et al., 2026; Liang et al., 2026).

3.2.3 The Fundamental Verifiability Trade-Off and Security Blindspot

Recasting all five representation formats through our six-tuple formalism $s = (\mathcal{A}, \mathcal{I}, \mathcal{C}, \mathcal{T}, \pi, \mathcal{E})$ exposes a foundational structural insight: as pre-execution verifiability increases monotonically from Natural Language to Formal Contracts, portability across heterogeneous LLM harnesses decreases monotonically.

Most critically, our analysis reveals a persistent security blindspot across all contemporary formats: no representation format verifies natural-language instructions $\mathcal{I}$ or activation conditions $\mathcal{A}$ prior to runtime invocation. While contract-based and structured formats rigorously constrain tool signatures ($\mathcal{T}$) and code AST execution ($\pi, \mathcal{E}$), they leave natural-language prose instructions completely uninspected. Consequently, adversarial payloads embedded within docstrings, markdown comments, or prompt instructions (e.g., indirect prompt injection (Maloyan & Namiot, 2026; Wang et al., 2026h)) bypass static code scanners and schema linters entirely, allowing malicious skills to hijack agent decision-making at execution time without triggering a single syntactic or typing error.

3.3 Skill Storage and Memory Architecture

An agent’s operational expertise is stored within distinct temporal memory tiers to maintain context efficiency, prevent cognitive overload, and enable lifelong capability persistence across multi-session deployments (Sumers et al., 2024; Packer et al., 2023; Zhang et al., 2026d). A systematic architectural comparison of procedural memory frameworks across storage tiers, index structures, and eviction policies is presented in Table 4.

3.3.1 Hierarchical Memory Tiering: From Episodic Traces to Procedural Assets

Drawing inspiration from cognitive architectures (Sumers et al., 2024) and operating systems (Packer et al., 2023), modern agent frameworks partition memory into distinct functional tiers along an *experience compression continuum* (Zhang et al., 2026d):

- **Episodic Memory ($\mathcal{H}$, 1:1 Compression):** Stores raw, chronological execution traces, observation histories, and dialogue interactions $\tau = (x_1, a_1, r_1, \dots, x_H)$ (Cao et al., 2026; Park et al., 2023). While episodic traces

Table 4: **Storage & Memory Architecture (§3.3).** Comparison of procedural skill memory and virtual storage architectures ($n = 11$ systems). Sorted primarily by **Eviction / compression policy** (*Virtual OS paging* $\rightarrow$ *Utility-aware curation* $\rightarrow$ *Semantic trace compression*). Demonstrates how memory systems address context overflow, retrieval pollution, skill technical debt, and catastrophic forgetting. All 10 systems implement lifelong multi-session persistence.

System	Core Contribution	Primary Tier	Index Structure	Eviction / Compression Policy	Failure Mode Addressed
<i>Virtual OS Paging & Mutable State (Bounded $O(1)$ Memory Management)</i>					
MemGPT Packer et al. (2023)	OS-level virtual context paging	virtual paging	OS page table	LLM-controlled paging	context overflow
SKILL.state Badhe et al. (2026)	Mutable execution state runtime	procedural / state	JSON state schema	Ephemeral reasoning disposal ($\Delta\Sigma$)	context bloat / $O(T^2)$ cost
<i>Utility-Aware Curation (Pruning Low-Value, Redundant, or Polluting Skills)</i>					
SkillOps Pu et al. (2026)	5-dimension library health maintenance	procedural	topological graph (HSEG)	utility pruning (5-dim score)	skill technical debt
MEMP Fang et al. (2026)	Learnable lifelong procedural memory	procedural	dense vector	utility pruning	retrieval pollution
SkillOS Ouyang et al. (2026)	Utility-aware skill pool curation	procedural	dense vector (MiniLM)	RL-based repo curation & pruning	retrieval pollution
MemSkill Zhang et al. (2026c)	Learnable memory operation skills	procedural	dense vector (Qwen3)	Learnable skill-based pruning & consolidation	retrieval pollution
SkillBank Tu et al. (2026)	Dual-granularity agentic skill bank	hierarchical	dense vector	Hindsight utility pruning (D2Skill dual-granularity)	retrieval pollution
<i>Semantic Trace Compression (Abstracting Dialogue & Trajectories into Compacting Abstractions)</i>					
ContinualMem Hu et al. (2026b)	Memory stability–plasticity analysis	procedural	dense vector	semantic compression	catastrophic forgetting
ReMe Cao et al. (2026)	Evolving experience pool memory	procedural	dense vector (Qwen3)	semantic compression	context overflow
Memento Zhou et al. (2026)	Agent-designing procedural memory	procedural	hybrid (BM25 + vector)	semantic compression	context overflow
ExperienceSpectrum Zhang et al. (2026d)	Unifying compression spectrum (5–1000 $\times$)	hierarchical	multi-level spectrum	semantic compression (5–1000 $\times$)	context overflow

Notes · Column Value Definitions. **Core Contribution:** Primary architectural or algorithmic innovation. **Primary Tier:** Target memory abstraction (*procedural*: executable skill knowledge; *episodic*: fine-grained conversation and rollout logs; *virtual paging*: OS-level virtual memory hierarchy; *hierarchical*: multi-level memory structures). **Index Structure:** Retrieval data structure (*dense vector*: vector embedding similarity; *topological graph*: dependency or structural graph; *OS page table*: working vs. archival page tables; *hybrid*: BM25 + dense embedding; *multi-level spectrum*: unified compression spectrum). **Eviction / Compression Policy:** Saturation mitigation mechanism (*LLM-controlled paging*: agent-directed virtual memory page swaps to archival storage via explicit function calls; *utility pruning*: pruning low-utility, redundant, or broken skills; *semantic compression*: compacting dialogue and trajectories into semantic abstractions). **Failure Mode Addressed:** Target operational bottleneck (*context overflow*: prompt token saturation; *retrieval pollution*: stale or redundant skills degrading top- k precision; *skill technical debt*: persistent library-level defects such as interface drift or missing validation; *catastrophic forgetting*: memory-level interference across lifelong task distributions).

preserve full operational granularity, their linear context footprint causes rapid token saturation and high inference latency if loaded directly.

- **Procedural Memory** ($\mathcal{M}_{\text{proc}}$, $10\text{--}100\times$ **Compression**): The persistent, non-parametric storage substrate for modular, executable skills $s = (\mathcal{A}, \mathcal{I}, \mathcal{C}, \mathcal{T}, \pi, \mathcal{E})$ (Fang et al., 2026; Zhang et al., 2026d). Procedural memory decouples high-level operational workflows from raw execution mechanics, storing standardized SOPs, tool orchestration pipelines, and verified scripts that can be invoked across disparate task instances without retaining episodic noise.
- **Virtual OS Paging and Mutable Execution State** ($C_t \subset \mathcal{M}$): To bridge fixed context windows with massive external memory, architectures such as MemGPT (Packer et al., 2023) implement OS-inspired virtual memory management, dynamically paging memory blocks between active working context and archival storage. Moving beyond unconstrained text transcripts, SKILL.state (Badhe et al., 2026) formalizes runtime working context as an explicit, mutable execution state $\Sigma_t \in \Sigma$. By projecting observations into structured JSON patches ($\Delta\Sigma_t$) and immediately discarding ephemeral intermediate reasoning R_t , SKILL.state achieves a strictly bounded $O(1)$ working prompt footprint and linear $O(T)$ cumulative token complexity across extended horizons.
- **Symbolic and Rule Memory** ($1000\times$ **Compression**): Highly consolidated global heuristics, system constraints, and negative constraints extracted from lifelong experience (Zhang et al., 2026d).

3.3.2 Mathematical Formalization of the Memory Update and Curation Operator

As an agent interacts with sequential environments, its procedural memory bank $\mathcal{M}_t$ evolves dynamically. We formalize the procedural memory update operator $\mathbb{U}$ as:

$$\mathcal{M}_{t+1} = \mathbb{U}(\mathcal{M}_t, \tau_t, \mathcal{F}_t) = (\mathcal{M}_t \setminus \mathbb{E}_v(\mathcal{M}_t)) \cup \{s^*\} \quad (11)$$

where τ_t is the recent execution trace, $\mathcal{F}_t$ is environmental feedback, s^* is a newly consolidated candidate skill, and $\mathbb{E}_v(\mathcal{M}_t) \subset \mathcal{M}_t$ is the set of evicted or pruned skills identified by the memory curation policy.

To determine which skills should be retained or evicted, frameworks evaluate an empirical utility function $U(s)$:

$$U(s) = \mathbb{E}_{(x,g) \sim \mathcal{D}} [R(s(x), g) - R(\pi_{\text{base}}(x), g)] - \lambda \cdot \text{Cost}(s) \quad (12)$$

where the first term represents the *counterfactual performance gain* of injecting skill s over a baseline policy π_{base} (Tu et al., 2026; Ouyang et al., 2026), and $\text{Cost}(s)$ incorporates retrieval latency, token footprint, and execution overhead. Skills whose empirical utility falls below an eviction threshold ($U(s) < \gamma_{\text{prune}}$) or whose activation frequency exhibits temporal decay are systematically purged by $\mathbb{E}_v$ to maintain library solvency.

3.3.3 Mitigating Memory Failures: The Four Operational Bottlenecks

Unconstrained accumulation of skills across lifelong deployments triggers four fundamental memory failure modes:

1. **Context Overflow and Saturation:** Storing unbounded conversational histories rapidly exhausts LLM context windows. Systems like ReMe (Cao et al., 2026), Memento (Zhou et al., 2026), and ExperienceSpectrum (Zhang et al., 2026d) deploy reflective summarization agents to continuously compress raw episodic trajectories into compact procedural cards and dense vector embeddings, achieving $5\times\text{--}1000\times$ token reductions.
2. **Retrieval Pollution and Confusability:** As the skill library $|\mathcal{M}|$ grows beyond hundreds of files, dense embedding spaces become densely crowded. Near-duplicate docstrings or overlapping descriptions create semantic confusability, causing retrievers to return sub-optimal or conflicting skills that degrade task success (Fang et al., 2026; Liu et al., 2026i). MEMP (Fang et al., 2026) and SkillIOS (Ouyang et al., 2026) resolve this by coupling learnable utility tracking with MiniLM-based sentence embedding cosine filters, pruning stale or low-utility routines. In reinforcement learning, SkillBank / D2Skill (Tu et al., 2026) organizes skills into dual granularities (high-level task skills and low-level step skills), applying hindsight utility pruning based on paired rollout performance gaps.

3. **Skill Technical Debt and Dependency Drift:** Over time, underlying tool APIs mutate, software packages deprecate, and sub-skills become mutually incompatible. SkillOps (Pu et al., 2026) models these interactions as a *Hierarchical Skill Ecosystem Graph* (HSEG), monitoring library health across five dimensions (utility, compatibility, risk, validation, redundancy) to actively refactor stale dependencies and resolve skill technical debt. Similarly, SkillClone (Zhu et al., 2026) deploys multi-modal clone detection to consolidate redundant code implementations.
4. **The Stability–Plasticity Dilemma and Catastrophic Forgetting:** While non-parametric memory is often assumed to solve continual learning without weight retraining, Hu et al. (2026b) empirically demonstrate that memory-augmented agents face a severe stability–plasticity trade-off. Fine-grained procedural skills that maximize forward transfer on novel tasks simultaneously introduce severe backward interference and memory-level forgetting on historical tasks, proving that external memory requires structured modular isolation rather than flat pooling.

3.4 Skill Retrieval and Dynamic Routing

As procedural memory repositories scale from dozens of local scripts to tens of thousands of ecosystem-wide packages ($|\mathcal{M}| > 10^4$), exposing the entire skill library in the prompt context becomes computationally intractable (Zheng et al., 2026; Liu et al., 2026i). Autonomous agents require dynamic retrieval and routing pipelines that selectively identify, rank, and route the optimal procedural skill (or composable sub-graph of skills) conditioned on the active execution state x_t and goal g . A systematic architectural comparison of representative retrieval and routing frameworks is presented in Table 5.

Table 5: **Retrieval & Dynamic Routing (§3.4).** Comparison of procedural skill retrieval and agent routing architectures ($n = 8$ systems). Sorted primarily by **Retrieval / routing paradigm** (*Retrieve-then-Rerank* $\rightarrow$ *Graph-Structured Search* $\rightarrow$ *Agentic Navigation* $\rightarrow$ *Dynamic Agent Routing*), then secondarily by **Scale evaluated** (descending). Demonstrates the trade-off between massive-scale top- k retrieval and structured compositional graph navigation.

System	Core Contribution	Retrieval / Routing Paradigm	Retrieval Granularity	Selection / Ranking Signal	Pollution / Redundancy Control
<i>Retrieve-then-Rerank (Dense Embedding Search & Listwise / Contrastive Reranking Pipelines)</i>					
SkillRet Cho et al. (2026)	Large-scale retrieve-then-rerank IR	Retrieve-then-Rerank	Atomic skill SOP	Dense vector + cross-encoder	Contrastive reranking
SkillRouter Zheng et al. (2026)	Full-text retrieve-and-rerank (1.2B pipeline, 80K skills)	Retrieve-then-Rerank	Atomic skill SOP	Semantic sim. + listwise loss	Metadata-only routing blindness (31–44% drop)
SkillRAG Su et al. (2026)	Hybrid BM25 skill retrieval	Retrieve-then-Rerank	Atomic skill SOP	BM25 + vector hybrid	None (top- k only)
<i>Graph-Structured Search (Traversing Structural Dependency & Documentation Graphs)</i>					
GraphOfSkills Liu et al. (2026b)	Dependency-aware structural retrieval	Graph-Structured Search	Sub-graph workflow	Structural dependency edges	Dependency constraints
GraSP Xia et al. (2026c)	Graph-structured skill compositions	Graph-Structured Search	Sub-graph workflow	Structural dependency edges	Dependency constraints
GraphSkill Wang et al. (2026b)	Documentation-guided skill graph	Graph-Structured Search	Sub-graph workflow	Structural dependency edges	Documentation hierarchy
<i>Agentic Navigation (Offline Directory Compilation & Active Tree Traversal)</i>					
CORPUS2SKILL Sun et al. (2026)	Offline distillation into navigable hierarchical directory	Agentic Navigation	Sub-graph workflow	Agentic tree drilling & backtracking	Flat retrieval saturation & context blindness
<i>Dynamic Agent Routing (Cross-Agent Orchestration via Skill Capability Profiles)</i>					
SkillOrchestra Wang et al. (2026f)	Cross-agent skill transfer routing	Dynamic Agent Routing	Agent capability profile	Cross-agent transfer utility	Policy transfer

Notes · Column Value Definitions. **Core Contribution:** Primary architectural or algorithmic IR/routing innovation. **Retrieval / Routing Paradigm:** Overall retrieval or selection architecture (*Retrieve-then-Rerank*: two-stage vector/sparse retrieval followed by cross-encoder or listwise reranking; *Graph-Structured Search*: structural dependency or data-flow graph traversal; *Agentic Navigation*: offline compilation of hierarchical directories traversed actively by the agent without vector DBs; *Dynamic Agent Routing*: routing queries across multi-agent pools based on skill profiles). **Retrieval Granularity:** Scope of retrieved procedural knowledge (*Atomic skill SOP*: independent self-contained skill documents; *Sub-graph workflow*: connected multi-skill execution graphs or directories; *Agent capability profile*: skill bundles characterizing agent specializations). **Selection / Ranking Signal:** Primary mathematical scoring function or structural criterion used to score and select skills. **Pollution / Redundancy Control:** Mitigation mechanism used to prevent near-duplicate skills, interface mismatches, or retrieval noise from degrading top- k precision.

3.4.1 Mathematical Formalization of the Two-Stage Retrieval-Routing Pipeline

Modern platforms structure skill discovery and invocation through a multi-stage selection pipeline:

1. **Stage 1: Candidate Generation (Sparse-Dense Hybrid Retrieval):** Given an active state x_t and goal g , the platform retrieves a top- K candidate set $\mathcal{S}_{\text{cand}} \subset \mathcal{M}$ using a combination of dense semantic similarity and lexical BM25 token matching (Su et al., 2026; Cho et al., 2026):

$$\mathcal{S}_{\text{cand}} = \text{Top-}K_{s \in \mathcal{M}} \left(\alpha \cdot \text{Sim}(f_\theta(x_t, g), e_s) + (1 - \alpha) \cdot \text{BM25}(g, \text{doc}(s)) \right) \quad (13)$$

where $\alpha \in [0, 1]$ balances keyword matching against latent semantic retrieval, and e_s is the pre-computed embedding of skill s .

2. **Stage 2: Cross-Encoder Reranking and Capability Gating:** A cross-encoder scoring network Rerank_ϕ evaluates the joint interaction between the full task prompt and full-text procedural instructions, selecting the optimal skill s^* subject to activation and permission constraints (Zheng et al., 2026; Li et al., 2026g):

$$s^* = \arg \max_{s \in \mathcal{S}_{\text{cand}}} \text{Rerank}_\phi(x_t, g, s) \quad \text{subject to} \quad \mathcal{A}(x_t, g) = 1 \wedge \mathcal{T}_s \subseteq \mathcal{T}_{\text{permitted}}(x_t) \quad (14)$$

3.4.2 Comparative Paradigms in Procedural Routing

The literature approaches procedural discovery across four distinct architectural paradigms:

1. Retrieve-then-Rerank Pipelines: Frameworks such as SkillRet (Cho et al., 2026), SkillRouter (Zheng et al., 2026), and SkillRAG (Su et al., 2026) formulate selection as a classic information retrieval problem. SkillRet (Cho et al., 2026) establishes a large-scale benchmark for skill retrieval, proving that off-the-shelf bi-encoders struggle to distinguish procedural subroutines without task-skill fine-tuning. In large-scale production registries ($n = 80\text{K}$ skills), SkillRouter (Zheng et al., 2026) deploys a compact 1.2B bi-encoder and listwise reranker, achieving high retrieval accuracy at sub-second GPU latency.

2. Graph-Structured Search and Dependency Traversal: Flat vector retrieval treats skills as independent, isolated documents, ignoring execution prerequisites. To resolve this, GraphOfSkills (Liu et al., 2026b), GraSP (Xia et al., 2026c), and GraphSkill (Wang et al., 2026b) structure skills into topological dependency graphs $G = (V, E)$. Directed edges E enforce that prerequisite setup routines (e.g., environment initialization, database connections) are automatically retrieved alongside core execution skills, pruning disconnected subgraphs and guaranteeing execution ordering.

3. Agentic Navigation (Tree Traversal without Vector DBs): Challenging the premise of embedding-based retrieval, CORPUS2SKILL (Sun et al., 2026) introduces a “Don’t Retrieve, Navigate” paradigm. Offline clustering compiles complex corpora into a hierarchical, navigable directory tree of skill summary cards. Rather than performing flat similarity lookup, the agent actively navigates the tree via CLI inspection, drilling down into relevant sub-trees and backtracking upon reaching dead ends, entirely avoiding vector database saturation.

4. Dynamic Multi-Agent Routing via Skill Profiles: In compound multi-agent systems, SkillOrchestra (Wang et al., 2026f) shifts routing from retrieving raw text to routing queries across heterogeneous specialized agents. By estimating cross-agent skill transfer utility, SkillOrchestra maps sub-tasks to the specific agent whose capability profile exhibits the highest historical completion probability.

3.4.3 Critical Methodological Weaknesses: The Metadata-Only Blindness

Our synthesis highlights a critical failure mode pervasive across commercial agent harnesses: **metadata-only routing blindness**. To conserve token costs and minimize indexing latency, platforms frequently index only tool names and one-line summary descriptions. However, empirical experiments by Zheng et al. (2026) across 80,000 skills reveal that hiding the full skill body causes a catastrophic substantial degradation in task completion in routing accuracy. Because semantically distinct routines frequently share near-identical high-level descriptions (e.g., “extract table from document”), full-text procedural instructions $\mathcal{I}$ are essential for disambiguating tool constraints and preventing execution failures.

3.5 Skill Composition and Synthesis

As user objectives scale from single-turn tool calls to long-horizon, multi-modal workflows, executing isolated individual skills becomes insufficient. Autonomous agent platforms require procedural composition architectures that combine, chain, and orchestrate multiple atomic skills into complex execution pipelines (Xia et al., 2026b;c; Zabounidis et al., 2026). A systematic comparison of procedural skill composition, DAG compilation, and multi-agent orchestration frameworks is provided in Table 6.

Table 6: **Orchestration & Execution Verification (§3.5 & §3.6).** Comparison of procedural skill composition, multi-agent orchestration, and execution verification architectures ($n = 6$ systems). Sorted primarily by **Orchestration paradigm** (*Dynamic Graph Topology* $\rightarrow$ *Single-Agent Skill Switching* $\rightarrow$ *Multi-Agent SOP Collaboration*). Demonstrates the trade-off between dynamic graph communication topologies, low-latency single-agent skill loading, and role-based multi-agent collaboration.

System	Core Contribution	Orchestration Paradigm	Communication Topology	Task Decomposition Mechanism	Verification & Error Recovery
SkillGraph Nie et al. (2026)	Multimodal graph topology evolution	Dynamic Graph Topology	Dynamic graph (MMGT)	Query-conditioned graph predictor	Policy gradient on edge log-probs
SingleAgentSkills Li (2026)	Compiling multi-agent into single-agent skill selection	Single-Agent Skill Switching	In-context prompt loading	Skill router / selector	Capacity phase transition (sharp drop at critical size)
MetaGPT Hong et al. (2024)	SOP-guided multi-agent assembly lines	Multi-Agent SOP Collaboration	Assembly line chain	Role-specific SOP prompts	Role-based code review & testing
ChatDev Qian et al. (2024)	Software development agent society	Multi-Agent SOP Collaboration	Assembly line chain	Role-specific SOP prompts	Role-based code review & testing
AgentVerse Chen et al. (2024)	Emergent multi-agent peer collaboration	Multi-Agent SOP Collaboration	Dynamic broadcast / peer chat	Conversational dialogue turn	Multi-agent peer critique / voting
AutoGen Wu et al. (2024)	Conversable multi-agent programming	Multi-Agent SOP Collaboration	Dynamic broadcast / peer chat	Conversational dialogue turn	Multi-agent peer critique / voting

Notes · Column Value Definitions. **Core Contribution:** Primary architectural or algorithmic orchestration/verification innovation. **Orchestration Paradigm:** Overall composition or multi-agent execution architecture (*Dynamic Graph Topology*: dynamically constructing query-conditioned inter-agent communication graphs; *Single-Agent Skill Switching*: single-agent architecture dynamically loading skill packages into prompt context without multi-agent overhead; *Multi-Agent SOP Collaboration*: societies of specialized LLM agents collaborating via role-based SOPs or conversational turns). **Communication Topology:** Information flow structure among execution nodes or agent roles (*Dynamic graph (MMGT)*: query-conditioned directed graph sampled via Bernoulli policy; *In-context prompt loading*: direct context replacement; *Assembly line chain*: sequential role handoffs; *Dynamic broadcast / peer chat*: conversational messaging across peer agents). **Task Decomposition Mechanism:** How a complex user goal is partitioned into manageable sub-tasks. **Verification & Error Recovery:** Mechanism used to validate intermediate execution outputs and recover from runtime failures (*node 3.6*).

3.5.1 Formalizing Compositional Structures: From Pipelines to Typed DAGs

We formalize procedural composition under three foundational computational structures:

1. **Sequential Pipeline Composition:** Two skills s_1 and s_2 are composed sequentially ($s_2 \circ s_1$) such that the terminal output state of s_1 directly populates the input execution context of s_2 :

$$(s_2 \circ s_1)(x_t) = s_2(s_1(x_t)) = \pi_2(\pi_1(x_t, \mathcal{I}_1, \mathcal{T}_1), \mathcal{I}_2, \mathcal{T}_2) \quad (15)$$

subject to the strict compositional precondition constraint $\mathcal{E}_1 \subseteq \mathcal{C}_2$, ensuring that the intended effects of s_1 satisfy the applicability context of s_2 (Microsoft, 2024).

2. **Dynamic DAG Orchestration:** Complex multi-step reasoning compiles retrieved skills into a typed Directed Acyclic Graph $\mathcal{G}_{\text{comp}} = (\mathcal{V}_{\text{skills}}, \mathcal{E}_{\text{data}})$, where vertices $v_i \in \mathcal{V}_{\text{skills}}$ represent skill execution nodes and directed edges $e_{ij} \in \mathcal{E}_{\text{data}}$ encode data-flow and control dependencies (Xia et al., 2026b;c). In GraSP (Xia et al., 2026c), topological graph compilation enables *locality-bounded error recovery*: when an execution error occurs at node v_k , the repair operator invalidates only the topological descendant subgraph $\text{Desc}(v_k) \subset \mathcal{V}_{\text{skills}}$, reducing replanning complexity from global $O(N)$ full re-execution to localized $O(d^h)$ subtree repair.
3. **Hierarchical and Recursive Invocation:** In hierarchical composition, a high-level master planner delegates sub-goals to specialized sub-skills, isolating intermediate reasoning traces and preserving context budget (Du et al., 2024; Jiao et al., 2026). In self-improving reinforcement learning, recursive invocation allows skills to invoke themselves or ancestral routines to resolve nested sub-problems (Xia et al., 2026a).

3.5.2 Comparative Orchestration Paradigms: Single-Agent vs. Multi-Agent Societies

The literature diverges on whether procedural composition should be orchestrated within a single unified agent or distributed across multi-agent societies:

1. Single-Agent Dynamic Skill Switching: Rather than spawning multiple conversational agents, single-agent architectures maintain a single centralized planner that dynamically loads and unloads modular skill packages into its working context on-demand (Li, 2026). Li (2026) demonstrate that compiling multi-agent systems into single-agent dynamic skill loaders substantially reduces inference token consumption and eliminates multi-agent conversational latency while maintaining competitive reasoning accuracy on standard benchmarks.

2. Multi-Agent SOP Collaboration and Assembly Lines: To tackle massive software development and enterprise workflows, frameworks such as MetaGPT (Hong et al., 2024), ChatDev (Qian et al., 2024), AgentVerse (Chen et al., 2024), and AutoGen (Wu et al., 2024) distribute procedural expertise across specialized role-playing agents (e.g., Product Manager, Software Architect, Code Reviewer, QA Engineer). In MetaGPT (Hong et al., 2024) and ChatDev (Qian et al., 2024), agents coordinate via standardized Standard Operating Procedures (SOPs) structured as assembly-line chains, passing structured design artifacts and performing role-based peer reviews to eliminate circular hallucinations.

3. Dynamic Graph Topology Evolution: Bridging fixed pipelines and unconstrained chat, SkillGraph (Nie et al., 2026) introduces a Multimodal Graph Topology (MMGT) where inter-agent communication channels are dynamically predicted conditioned on the user query. The agent society evolves its internal communication graph using policy gradients computed over edge selection log-probabilities, learning optimal task-conditioned routing topologies autonomously.

4. Symbolic Planning Grounded in Deep Reinforcement Learning: In continuous control and embodied manipulation, SCALAR (Zabounidis et al., 2026) unifies high-level LLM symbolic planning with low-level deep RL grounding. The language model proposes compositional skill hypotheses (preconditions, effects, reward functions), while deep RL policies ground motor control execution, feeding back execution failure tracebacks to iteratively refine the LLM’s symbolic skill definitions.

3.5.3 Critical Methodological Weaknesses: The Single-Agent Capacity Phase Transition

While single-agent skill loading is highly token-efficient, Li (2026) uncover a fundamental structural limitation: the **Capacity Phase Transition**. As the number of available candidate skills exceeds a critical library threshold (N_{crit}), single-agent prompt routers suffer a sharp, non-linear collapse in selection accuracy due to attention dilution and semantic interference. Beyond this critical threshold, partitioned multi-agent architectures (which enforce strict context isolation and role boundaries) consistently outperform monolithic single-agent systems despite their higher token overhead.

3.6 Skill Execution, Verification, and Repair

Once procedural skills are retrieved and composed, runtime execution governs active invocation, boundary enforcement, environment state transformation, and automated exception recovery (Chen et al., 2026c; Li et al., 2026g).

3.6.1 Conditional Activation and Capability-Based Permission Gating

A skill $s = (\mathcal{A}, \mathcal{I}, \mathcal{C}, \mathcal{T}, \pi, \mathcal{E})$ activates conditionally based on the agent’s current state $x_t \in \mathcal{X}$ and active task goal $g \in \mathcal{G}$ via the activation function $\mathcal{A}(x_t, g) \rightarrow \{0, 1\}$ (Zheng et al., 2026; Li et al., 2023). Crucially, to prevent unauthorized execution, privilege escalation, and confused-deputy attacks, activation is strictly bounded by a capability-based permission model (Li et al., 2026g):

$$\mathcal{A}(x_t, g) = 1 \implies \mathcal{T}_s \subseteq \mathcal{T}_{\text{permitted}}(x_t) \quad (16)$$

A skill is permitted to fire only if its declared tool interfaces $\mathcal{T}_s$ represent a strict subset of the tools authorized in the active execution context $\mathcal{T}_{\text{permitted}}(x_t)$ (Li et al., 2026g; Yuan et al., 2024). If this subset constraint is violated

($\mathcal{T}_s \not\subseteq \mathcal{T}_{\text{permitted}}(x_t)$), the runtime intercepts the invocation prior to dispatch, blocking unauthorized tool access and preventing privilege escalation.

3.6.2 Runtime Execution and Sandboxed State Transitions

When activation and permission constraints are satisfied, the skill’s execution policy π executes procedural tool invocations under applicability constraints $\mathcal{C}$, inducing a state transition (Zheng et al., 2026):

$$x_{t+1} = s(x_t) = \pi(x_t, \mathcal{I}, \mathcal{T}) \quad (17)$$

In traditional conversational runtimes (e.g., ReAct), execution history is maintained by continually appending observations, actions, and reasoning traces to the context prompt, triggering quadratic cumulative token consumption $\sum_{t=1}^T |P_t| = O(T^2)$ and severe context degradation over extended trajectories (Badhe et al., 2026). To resolve this systems-level scaling bottleneck, SKILL.state (Badhe et al., 2026) replaces append-only conversational history with an explicit, mutable execution state Σ_t . At step t , the LLM receives the compact tuple $(\mathcal{P}, \Sigma_t, O_t)$ and generates intermediate reasoning R_t , a validated state patch $\Delta\Sigma_t$, and action a_t . Crucially, intermediate reasoning R_t is discarded immediately upon applying the validated state transition $\Sigma_{t+1} \leftarrow \Sigma_t \oplus \Delta\Sigma_t$, maintaining a bounded $O(1)$ prompt footprint per step and substantially reducing cumulative token consumption across long-horizon procedural benchmarks.

Because skills execute arbitrary shell commands, code snippets, and API mutations, execution must be isolated within strict sandboxed containers (Chen et al., 2026c; Marchand et al., 2026). However, empirical stress-testing by Marchand et al. (2026) demonstrates that frontier LLMs exhibit non-trivial container breakout capabilities when environment misconfigurations exist, emphasizing that software sandboxing must be coupled with runtime syscall filtering and network egress restrictions.

3.6.3 Runtime Verification and Invariant Checking

To verify execution integrity beyond static type checking, frameworks employ three distinct verification paradigms:

- **Contract-Based Invariant Verification:** Systems like ContractSkill (Lu et al., 2026b) evaluate formal pre-conditions and post-conditions ($V_{\text{pre}}(x_t) = 1$ and $V_{\text{post}}(x_{t+1}) = 1$) around execution steps. When an invariant is violated, the verifier synthesizes a structured counterexample that pinpoints the failing state transition.
- **Tool-Interactive Runtime Critique:** Frameworks such as CRITIC (Gou et al., 2024) and Self-Refine (Madaan et al., 2023) deploy secondary verifier tools (e.g., code linters, unit test runners, web search validators) to evaluate intermediate execution outputs dynamically, triggering introspective self-correction before committing results to persistent memory.
- **Formal Constraint Proving:** In safety-critical pipelines, static-dynamic analyzers like SemiA (Wen et al., 2026) evaluate execution traces against Datalog safety policies in the Skill Description Language (SDL), ensuring no tainted variables reach sensitive sink APIs.

3.6.4 Failure Attribution and Locality-Bounded Error Repair

When runtime exceptions or contract violations occur, naive execution architectures trigger full-system replanning from scratch, incurring severe latency and token overhead. Modern architectures deploy structured failure attribution and localized repair mechanisms:

- **Structural Failure Attribution:** In long-horizon web and software tasks, SkillTracer (Li et al.) analyzes execution tracebacks and DOM state diffs to attribute failures to specific upstream sub-skills, distinguishing semantic reasoning flaws from transient API errors.
- **Locality-Bounded DAG Repair:** In graph-orchestrated workflows, GraSP (Xia et al., 2026c) implements topological localized repair. Instead of restarting the entire workflow ($O(N)$ replanning complexity), the repair operator $\mathbb{R}_{\text{DAG}}$ isolates and replans only the reachable descendant subgraph $\text{Desc}(v_k)$ of the failing node v_k , reducing error recovery complexity to $O(d^h)$ where d is graph branching factor and h is subtree depth.

- **Iterative Error Traceback Reflection:** As pioneered in Voyager (Wang et al., 2024) and Trace2Skill (Ni et al., 2026), runtime exceptions (e.g., Python tracebacks, syntax errors, missing dependencies) are fed directly into the model’s prompt context, allowing the LLM to iteratively repair the buggy procedural code and re-execute in a sandboxed trial loop.

3.7 Skill Adaptation and Evolution

In open-ended and non-stationary environments, static procedural assets inevitably suffer from performance degradation due to API deprecation, environment drift, and newly encountered edge cases. Autonomous agents require mechanisms to adapt and evolve their capabilities across two distinct temporal horizons: short-term in-context adaptation during active inference, and long-term lifelong evolution across multi-session deployments (Yang et al., 2026b; Zhang et al., 2026b;e).

3.7.1 Short-Term In-Context Adaptation and Dynamic Refinement

Short-term adaptation occurs within a single task session without modifying persistent storage:

- **Verifier-Guided Dynamic Parameter Tuning:** In specialized domains such as electronic design automation, Trace2Skill-EDA (Du & Pinckney, 2026) demonstrates that coupling retrieved skills with dense simulator verifier feedback allows the agent to dynamically adjust script parameters and tool arguments on-the-fly, substantially improving EDA task completion under simulator verification.
- **Interactive Introspective Self-Correction:** Architectures such as CRITIC (Gou et al., 2024) and Self-Refine (Madaan et al., 2023) implement test-time feedback loops. When intermediate verification yields execution warnings or sub-optimal outputs, reasoning skills analyze execution traces, diagnosing syntax errors and generating refined plans prior to terminal action dispatch.

3.7.2 Long-Term Evolution and Reinforcement Learning

Long-term evolution permanently updates the agent’s procedural memory bank $\mathcal{M}$ and high-level decision policies:

- **Non-Parametric Policy Gradients in Text Space:** In Skill-Pro (Mi et al., 2026), skills are formalized as options within a Skill-MDP $M_\Omega = (\mathcal{S}, \mathcal{A}, \Omega, P, R, \gamma)$. Non-parametric Proximal Policy Optimization extracts natural language semantic gradients ($g_i = \nabla_{\text{sem}}(\tau_i, \omega)$) from trajectory batches, iteratively refining textual instructions and pruning low-advantage routines.
- **Bi-Level Optimization and Verifiable RL:** In Skill-R1 (Vishe et al., 2026), skill evolution is framed as bi-level reinforcement learning: the inner loop optimizes execution parameters against verifiable task rewards, while the outer loop updates library membership and instruction formulation. Similarly, ARISE (Li et al., 2026e) grounds intrinsic skill evolution within hierarchical reinforcement learning, optimizing option policies against intrinsic exploration rewards.
- **Lifelong Meta-Skill Evolution:** Frameworks like AutoSkill (Yang et al., 2026b) and SkillClaw (Ma et al., 2026c) deploy higher-order meta-skills $m : \mathcal{S} \rightarrow \mathcal{S}$ that inspect historical rollout logs $\mathcal{F}_t$, executing automated refactoring, interface standardization, and redundant skill pruning. In context engineering, MetaAgent (Ye et al., 2026) optimizes procedural prompt structures to maximize lifelong capability transfer.

3.7.3 Mathematical Formalization of Policy-Skill Co-Evolution

Recent frameworks demonstrate that evolving skills in isolation is sub-optimal; an agent’s high-level decision policy π_{dec} and procedural skill library $\mathcal{M}$ must co-evolve synergistically (Zhang et al., 2026b; Wu et al., 2026a). We formalize the policy-skill co-evolution objective as:

$$\max_{\pi_{\text{dec}}, \mathcal{M}} \mathbb{E}_{\tau \sim (\pi_{\text{dec}}, \mathcal{M})} [R(\tau)] \quad \text{subject to} \quad s_{t+1} = m(s_t, \mathcal{F}_t) \wedge \mathcal{A}_{t+1} = m(\mathcal{A}_t, \mathcal{F}_t) \quad (18)$$

The co-evolution proceeds via alternating optimization:

$$\pi_{\text{dec}}^{(t+1)} \leftarrow \text{PolicyUpdate}(\pi_{\text{dec}}^{(t)}, \mathcal{M}^{(t)}), \quad \mathcal{M}^{(t+1)} \leftarrow \text{SkillUpdate}(\mathcal{M}^{(t)}, \pi_{\text{dec}}^{(t+1)}, \mathcal{F}_t) \quad (19)$$

In CoEvoSkills (Zhang et al., 2026b), an informationally isolated surrogate verifier is co-evolved alongside the skill generator, providing adversarial checks that prevent the co-evolutionary loop from overfitting to spurious execution shortcuts.

3.7.4 Critical Methodological Weaknesses: Co-Evolutionary Drift and Benchmark Overfitting

Our synthesis uncovers two critical open vulnerabilities in lifelong skill evolution:

1. **Policy-Skill Co-Adaptation Drift:** When high-level planners and skill banks co-evolve without strict interface contracts, the decision policy π_{dec} frequently develops brittle dependencies on idiosyncrasies of specific skill drafts (Zhang et al., 2026b; Wu et al., 2026a). Consequently, upgrading or refactoring an upstream skill can silently break downstream planner logic, causing catastrophic regression across previously solved tasks.
2. **Evaluation Contamination in Lifelong Benchmarks:** As highlighted by SkillFlow (Zhang et al., 2026e), many lifelong learning benchmarks evaluate agent evolution on task distributions with identical underlying tool APIs. This shared environment bias obscures whether the evolutionary loop is learning generalized procedural abstractions or merely overfitting to deterministic API calling sequences.

3.8 Empirical Evaluation and Benchmark Suites

Rigorous empirical evaluation is essential to establish benchmarks for system reliability, token efficiency, continual adaptation, and safety alignment in skill-augmented language agents (Li et al., 2026d; Han et al., 2026; Liu et al., 2026i). While early agent evaluations relied on single-turn API matching, modern benchmarks evaluate the procedural skill lifecycle across seven foundational dimensions:

3.8.1 The Seven Foundational Evaluation Dimensions

1. **Correctness and Task Completion ($\text{Acc}_{\mathcal{M}}$):** Measures the end-to-end pass rate on complex environments under deterministic verification suites (Li et al., 2026d; Han et al., 2026). In SkillsBench (Li et al., 2026d) (87 tasks across 8 domains evaluated across 18 model-harness configurations), injecting curated skills provides substantial pass rate gains across diverse model and harness backbones. In software engineering, SWE-SkillsBench (Han et al., 2026) pairs 49 public SWE skills with 565 real-world GitHub task instances, using execution-based acceptance criteria to isolate the marginal utility of procedural skills.
2. **Lifelong Continual Learning:** Evaluates the agent’s ability to discover, consolidate, and reuse procedural skills across multi-session trajectories without catastrophic forgetting (Zhong et al., 2026; Zhang et al., 2026e). SkillLearnBench (Zhong et al., 2026) benchmarks continual skill generation across 20 verified real-world tasks in 15 sub-domains, revealing that while continual learning improves over no-skill baselines, backward interference across evolving libraries remains an open challenge.
3. **Robustness Under Open-Registry Retrieval Noise:** Benchmarks the resilience of skill selection when the agent must autonomously retrieve routines from massive, uncurated registries ($n > 10^4$) containing noisy or adversarial distractors (Liu et al., 2026i). Liu et al. (2026i) audit 34,000 real-world skills and reveal a severe *in-the-wild degradation*: while force-loading oracle skills provides high baseline utility, autonomous open-pool retrieval triggers severe performance degradation due to distractor noise, with many trajectories failing to retrieve required capabilities.
4. **Composability and Graph Scalability:** Verifies the joint execution success of multi-skill composition graphs as Directed Acyclic Graph (DAG) depth and branching factor increase (Shen et al., 2024; Xia et al., 2026c). On TaskBench (Shen et al., 2024) (1000+ tool automation graphs), execution accuracy drops exponentially with graph depth due to compounding interface mismatches and unhandled intermediate errors.

5. **Token Economy and Serving Latency:** Evaluates the computational cost of skill augmentation, incorporating load-time context consumption, inference latency, and runtime memory footprint (Gao et al., 2026; Chen et al., 2026a; Badhe et al., 2026). While SkillReducer (Gao et al., 2026) optimizes token consumption through automated schema compression, SKILL.state (Badhe et al., 2026) demonstrates on SkillExecBench (warehouse inventory and software repository relational graphs) and interactive suites (InterCode CTF, Sierra τ -Bench) that replacing append-only history with mutable state reduces token costs from quadratic $O(T^2)$ to linear $O(T)$ while improving task pass rates.
6. **Interpretability and Usability:** Measures whether procedural specifications serve as transparent capability disclosures for human operators (Wen, 2026). As demonstrated by Wen (2026), human comprehension of agentic skills requires explicit pre-condition disclosures and structured markdown formatting to enable predictable human-agent collaboration.
7. **Safety, Harm Weaponization, and Threat Resistance:** Quantifies the vulnerability of autonomous agents to malicious, poisoned, or unvetted third-party skill files (Jiang et al., 2026b; Badhe & Tiwari, 2026). In HarmfulSkillBench (Jiang et al., 2026b) (an audit of 98,440 real-world registry skills identifying weaponized skills in public registries), pre-installed harmful skills bypass standard LLM safety guardrails, substantially increasing harm compliance under implicit malicious queries.

3.8.2 Critical Empirical Insights: When Skills Do Not Help

Our synthesis highlights a critical, often-overlooked negative result in the agent evaluation literature: the Environment-Feedback Bandwidth Hypothesis (Chacko et al., 2026). In offensive cybersecurity Capture-the-Flag (CTF) challenges across 120+ tasks, Chacko et al. (2026) discover that injecting procedural skills yields zero statistically significant performance gain over raw tool-use baselines (an insignificant 8.9 percentage point spread, $p = 0.71$).

The authors demonstrate that when an agent interacts with high-bandwidth, schema-validated environments (e.g., interactive CLI shells that return immediate stderr tracebacks and diagnostic exit codes), the environment itself supplies the procedural feedback loop required for self-correction. In such high-feedback domains, static procedural skill guidelines provide redundant guidance while consuming prompt context budget, proving that the marginal utility of externalized skills is inversely proportional to environmental feedback bandwidth.

3.9 Security, Governance, and Market Defenses

Because procedural agent skills execute systems-level commands, access local filesystems, and interact with authenticated APIs, third-party skill distribution introduces critical security vulnerabilities across the entire agent supply chain (Li et al., 2026g,f; Badhe & Tiwari, 2026). As formalized in our six-tuple abstraction $s = (\mathcal{A}, \mathcal{I}, \mathcal{C}, \mathcal{T}, \pi, \mathcal{E})$, every vulnerability class maps directly to a specific procedural component, with the unverified components ($\mathcal{I}$ and $\mathcal{A}$) serving as primary conduits for zero-click privilege escalation. To systematically quantify lifecycle-wide risks beyond isolated execution prompts, SkillSec-Eval (Badhe & Tiwari, 2026) establishes a comprehensive threat benchmark across 327 real-world enterprise skills, revealing that undefended pipelines suffer from widespread malicious admission, Sybil retrieval hijacking, and planner susceptibility to fake social-proof recommendations.

3.9.1 Threat Vectors and Offensive Exploitations

An exhaustive taxonomy and comparison of adversarial threat vectors targeting procedural skills is detailed in Table 7.

The literature reveals that offensive attack vectors exploit four structural vulnerabilities:

1. **Indirect Prompt Injection via Unverified Instructions ($\mathcal{I}$):** Attackers embed adversarial natural language payloads within procedural documentation, Markdown headers, HTML comments, or multimodal diagram layers (Wang et al., 2026h; Schmotz et al., 2025a; Jia et al., 2026b). When the host planner loads the skill into context, these injected instructions override user objectives, hijacking high-privilege tools ($\mathcal{T}$) without triggering static code alarms.
2. **Backdoor and Trojan Execution Policies ($\pi, \mathcal{A}$):** Malicious actors deploy split encrypted code routines (SkillTrojan (Feng et al., 2026)) or model-in-skill weight poisoning (BadSkill (Tie et al., 2026)), embedding

Table 7: **Threat Vectors & Offensive Exploitations (§3.9.1).** Comparison of offensive security attacks and vulnerability studies ($n = 18$ systems). Sorted primarily by **Threat vector class** (*Indirect Prompt Injection* $\rightarrow$ *Backdoor / Trojan* $\rightarrow$ *Credential Exfiltration* $\rightarrow$ *Ecosystem & Supply Chain Risk*). Demonstrates how third-party skill packaging creates new attack surfaces across authoring, marketplace distribution, and runtime execution.

System / Attack	Core Contribution	Threat Vector Class	Target Component	Injection Surface / Channel	Target Lifecycle Stage	Compromise Impact
HiddenCommentInject Wang et al. (2026h)	invisible HTML comment prompt injection	Indirect Prompt Injection	$\mathcal{I}$	HTML / Markdown comments	Authoring	Goal hijacking
PromptInject Schmotz et al. (2025a)	Markdown instruction prompt injection	Indirect Prompt Injection	$\mathcal{I}$	Natural language prose	Authoring	Goal hijacking
SeeingsNot Jia et al. (2026b)	Multimodal hidden instruction attack	Indirect Prompt Injection	$\mathcal{I}$	Multimodal prose	Authoring	Goal hijacking
PhantomSkill Lin & Yu (2026)	VulMask auxiliary script injection	Indirect Prompt Injection	$\mathcal{I}$	Code body / auxiliary script	Authoring	Goal hijacking
SkillTrojan Feng et al. (2026)	Split encrypted backdoor composition	Backdoor / Trojan	π	Code body	Authoring	Sandbox escape
BadSkill Tie et al. (2026)	Model-in-skill weight poisoning	Backdoor / Trojan	π	Model weights in skill	Authoring	Sandbox escape
SkillAttack Duan et al. (2026)	Automated red-teaming path refinement	Backdoor / Trojan	$\mathcal{I}, \mathcal{A}$	Natural language prose	Marketplace	Goal hijacking
DynamicMalicious Chen et al. (2026d)	Dynamic runtime logic injection	Backdoor / Trojan	π	Natural language prose	Execution	Goal hijacking
CredentialLeakage Chen et al. (2026f)	Empirical study of credential exfiltration	Credential Exfiltration	$\mathcal{T}, \pi$	Code body	Execution	Data exfiltration
MalTool Hu et al. (2026c)	Malicious code embedding in tool scripts	Credential Exfiltration	$\mathcal{T}, \pi$	Code body	Marketplace	Data exfiltration
MaliciousAgentSkills Holzbauer et al. (2026)	Repository-aware security measurement	Credential Exfiltration	$\mathcal{T}, \pi$	Code body	Marketplace	Data exfiltration
SkillSec-Eval Badhe & Tiwari (2026)	Lifecycle-aware threat taxonomy & evaluation ($n = 327$ skills)	Lifecycle-Wide Threat Suite	$\mathcal{A}, \mathcal{I}, \mathcal{C}, \mathcal{T}, \pi$	Metadata, vector embeddings, planner prompts & code	All 5 stages (Admission to Evolution)	Retrieval hijack, planner manipulation & execution misuse
BehavioralIntegrity Wu et al. (2026b)	Empirical BIV audit of description-implementation gap ($n = 49, 943$ skills)	Ecosystem & Supply Chain Risk	$\mathcal{I}$ vs. π	Declared metadata vs. code implementation	Authoring & Registry	80.0% skills deviate from declared behavior
BenignComposition Xie et al. (2026b)	Multi-skill composition risk benchmark (SCR-Bench)	Ecosystem & Supply Chain Risk	$\mathcal{C}, \pi$	Multi-skill shared execution context	Execution	Capability escalation & auth confusion (33.6% ASR)
CloakDetonate Ji et al. (2026)	SKILLCLOAK SFS packing & structural obfuscation evasion	Ecosystem & Supply Chain Risk	π	Self-extracting packed skill code	Authoring & Execution	Bypasses 8 static/hybrid scanners (> 90% evasion)
HarmlessYetHarmful Hsu et al. (2026)	Neutral Prompting Attack (NPA) for hallucinated dependencies	Ecosystem & Supply Chain Risk	Semantically benign imagination prompts	Authoring	Unsafe package install via dependency name squatting	
SkillsAreNotIslands Jia et al. (2026a)	ASSCs & SBOM dependency graph audit ($n = 1.43\text{M}$ skills)	Ecosystem & Supply Chain Risk	Mixed skill-package-service dependency graph	Marketplace	Transitive dependency exposure & hidden inventory	
Proteus Zhou (2026)	Grey-box self-evolving red-team framework (PROTEUS)	Ecosystem & Supply Chain Risk	5-axis mutation across instructions, code & config	Authoring & Auditing	Bypasses SkillVetter & AI-Infra-Guard (40–90% ASR@5)	

Notes · Column Value Definitions. **Core Contribution:** Primary offensive discovery or vulnerability mechanism. **Threat Vector Class:** Primary exploitation category (*Indirect Prompt Injection*: hiding malicious instructions inside textual or commented documentation; *Backdoor / Trojan*: embedding dormant malicious triggers or poisoned model weights; *Credential Exfiltration*: stealing user API keys, tokens, or local data; *Ecosystem & Supply Chain Risk*: combinatorial multi-skill exploits, package squatting, or dependency poisoning). **Injection Surface / Channel:** Structural location where the payload is concealed. **Target Lifecycle Stage:** Point in the agent workflow where the attack is introduced. **Compromise Impact:** Primary operational consequence of successful exploitation.

poisoned LoRA tensors directly inside procedural repositories. Furthermore, CloakDetonate (Ji et al., 2026) introduces Self-Extracting File System (SFS) packing that achieves high evasion rates across standard static and hybrid security scanners.

3. **Credential Exfiltration and State Tampering ($\mathcal{T}, \pi$):** Large-scale audits by Chen et al. (2026f), Hu et al. (2026c), and Holzbauer et al. (2026) identify widespread exfiltration scripts embedded in community skills that harvest local `‘.env’` secrets, AWS credentials, and session tokens, transmitting them to external endpoints during tool execution.

4. Supply Chain Risks and Multi-Skill Compositional Escalation:

- *Description-Implementation Discrepancy:* A massive audit of 49,943 OpenClaw skills by Wu et al. (2026b) uncovers a widespread description-implementation gap where executable code deviates significantly from declared capability metadata.
- *Emergent Compositional Exploits:* In BenignComposition / SCR-Bench (Xie et al., 2026b), researchers demonstrate that skills which are entirely benign in isolation can combine in a shared execution context to create unauthorized capability escalations, achieving substantial attack success rates under emergent composition.
- *Dependency Squatting via Hallucination:* In HarmlessYetHarmful (Hsu et al., 2026), Neutral Prompting Attacks trick LLMs into generating non-existent package dependencies, enabling attackers to register malicious PyPI packages that are automatically installed by unsuspecting autonomous agents.
- *Agentic Software Supply Chains (ASSCs):* Jia et al. (2026a) analyze 1.43 million skills, demonstrating that recursive skill reuse creates complex, unmonitored dependency graphs with hidden package vulnerabilities.

3.9.2 Safety, Governance, and Marketplace Defenses

Representative defensive auditing frameworks, runtime guardrails, and marketplace governance platforms are compared in Table 8.

Table 8: **Safety, Governance & Market Defenses (§3.9.2).** Comparison of security auditing, runtime guardrail, and marketplace defense architectures ($n = 11$ systems). Sorted primarily by **Defense lifecycle stage** (*Pre-Admission Static Audit* → *Runtime Guardrail & Filtering* → *Multi-Agent Auditing* → *Marketplace Governance*).

System / Defense	Core Contribution	Defense Lifecycle Stage	Target Threat Defended	Serving Latency Overhead
SemiA Wen et al. (2026)	Constraint-guided formal skill auditing	Pre-Admission Static Audit	Backdoor / Trojan	Zero (offline audit)
SkillFortify Bhardwaj	Formal supply chain security analysis	Pre-Admission Static Audit	Malicious Code	Zero (offline audit)
SkillAuditSAST Lv et al. (2026)	Cross-file SAST security auditing	Pre-Admission Static Audit	Malicious Code	Low (SAST filter)
SkillSieve Hou & Yang (2026)	Hierarchical triage & static scanning	Pre-Admission Static Audit	Prompt Injection	Low (SAST filter)
SkillClone Zhu et al. (2026)	Multi-modal clone detection across 196K skills	Pre-Admission Static Audit	Clone Infringement	Zero (offline audit)
FlowGuard Etteib et al. (2026)	Attention & flow graph malware detection	Runtime Guardrail & Filtering	Malicious Code	Low (SAST filter)
RouteGuard Xiao et al. (2026)	Internal-signal runtime poisoning detection	Runtime Guardrail & Filtering	Prompt Injection	Medium-High (runtime judge)
STARS Zhang et al. (2026a)	Request-conditioned dynamic invocation audit	Runtime Guardrail & Filtering	Backdoor / Trojan	Medium-High (runtime judge)
SkillProbe Guo et al. (2026)	Multi-agent security auditing framework	Multi-Agent Auditing	Multi-Skill Combinatorial	Zero (offline audit)
SkillSec-Eval Badhe & Tiwari (2026)	Lifecycle-aware evaluation across 5 stages ($n = 327$ skills)	All 5 stages (Admission to Evolution)	Retrieval hijack, planner abuse & code exploits	Zero (offline lifecycle audit)
SecureAgentSkills Li et al. (2026f)	Modular filesystem security architecture	Marketplace Governance	Supply Chain Risk	Low (SAST filter)

Notes · Column Value Definitions. **Core Contribution:** Primary defensive, auditing, or governance innovation. **Defense Lifecycle Stage:** Operational point of defense deployment (*Pre-Admission Static Audit*: auditing skills prior to registry upload or loading; *Runtime Guardrail & Filtering*: active monitoring and filtering during skill invocation; *Multi-Agent Auditing*: multi-agent collaborative triage; *Marketplace Governance*: platform-level filesystem and provenance standards). **Target Threat Defended:** Primary offensive vector mitigated. **Serving Latency Overhead:** Computational latency introduced during runtime agent execution.

Defensive strategies operate across three coordinated architectural layers:

1. **Pre-Admission Static Auditing (Zero Latency Overhead):** To prevent malicious packages from entering registries, static vetting tools inspect AST structures and data flows offline. SemiA (Wen et al., 2026) synthesizes constraint-guided Datalog rules to prove safety invariants before registry admission. SkillFortify (Bhardwaj) implements SAT-based dependency resolution with capability sandboxing proofs, demonstrating strong detection performance and high classification precision. To address intellectual property theft and unauthorized repackaging, SkillClone (Zhu et al., 2026) deploys multi-modal clone detection across 196,000 skills, identifying copycat packages that propagate unpatched vulnerabilities.
2. **Dynamic Runtime Guardrails and Interceptors:** Because dynamic evasion and self-extracting loaders (CloakDetonate (Ji et al., 2026)) bypass static filters, runtime monitors actively inspect execution flows. RouteGuard (Xiao et al., 2026) tracks internal LLM activation signals to detect prompt injection attempts during skill loading, while STARS (Zhang et al., 2026a) executes request-conditioned dynamic invocation audits to verify that tool arguments conform to user intent.
3. **Collaborative Governance and Lifecycle Defense Suites:** To mitigate multi-skill composition risks, SkillProbe (Guo et al., 2026) deploys multi-agent red-teaming societies to test combinatorial skill interactions before runtime execution. At the platform level, SecureAgentSkills (Li et al., 2026f) establishes granular filesystem access control lists and cryptographic signature verification. Across the full procedural lifecycle, SkillSec-Eval (Badhe & Tiwari, 2026) evaluates 327 enterprise skills across 15 domains, proving that hybrid structural-semantic validation substantially reduces malicious admission and blocks unauthorized tool invocations, while exposing that traditional string-based taint tracking suffers from implicit information loss in LLM internal context.

4 Taxonomy of Skills

Procedural agent skills are modular packages of executable knowledge that augment large language model (LLM) agents at inference time. While existing literature often conflates general tool use, static system prompts, and dynamic procedural routines, we formalize a comprehensive taxonomy that categorizes procedural skills along five foundational dimensions: (1) instructional and Standard Operating Procedure (SOP) skills, (2) tool and API calling skills, (3) reasoning and planning skills, (4) domain-specific execution skills, and (5) collaborative and meta-skills. A comparative synthesis of representative skill architectures across these functional domains is presented in Table 9.

4.1 Instructional and SOP-Based Skills

Instructional and Standard Operating Procedure (SOP) skills represent procedural assets where capabilities are primarily encoded as natural language guidelines, structured checklists, and human workflow rules (Bi et al., 2026; Yuan et al., 2024). Unlike raw prompt engineering, instructional skills restrict an agent’s reasoning trajectory to verifiable, domain-tested operational steps.

4.1.1 Formalizing Procedural Task Execution

We formalize an instructional SOP skill as a state-to-state guidance mapping $s_{\text{task}} : \mathcal{X} \rightarrow \mathcal{X}$ that constrains action selection $\pi(a_t | x_t, \mathcal{I})$ to conform to declared procedural instructions $\mathcal{I}$. The skill activates conditionally via:

$$\mathcal{A}_{\text{task}}(x_t, g) = 1 \iff g \in \mathcal{G}_{\text{task}} \quad (20)$$

where $\mathcal{G}_{\text{task}}$ represents the set of supported goal descriptions matching the skill’s applicability context.

4.1.2 Comparative Authoring: From Manual Design to Automated Repository Mining

The literature approaches instructional skill creation across two distinct paradigms:

- **Automated Procedural Mining:** Historically, authoring SOPs required manual human curation, creating a severe operational bottleneck. To automate acquisition at scale, Bi et al. (2026) (ProceduralMining) introduce automated pipelines that mine open-source software repositories and enterprise runbooks, extracting reusable

Table 9: **Core Capabilities & Domain-Specific Skills (§4).** Comparison of procedural skill architectures across 5 primary functional and domain classes ($n = 16$ systems). Sorted primarily by **Skill functional domain** (*Instructional SOP* → *Tool & API Calling* → *Software Engineering* → *GUI & OS Navigation* → *Embodied & Robotics*). Demonstrates how individual skill representations adapt their action space granularity and environment feedback loop to solve distinct domain-specific bottlenecks.

System / Framework	Core Contribution	Skill Functional Domain	Action Space Granularity	Environment Feedback Loop	Domain Bottleneck Addressed
ProceduralMining Bi et al. (2026)	Large-scale procedural mining from repos	Instructional SOP (§4.1)	Mined open-source SOP checklists	Static repository mining (no loop)	Manual SOP authoring bottleneck
SkillAnalysis Ling et al. (2026)	Structural analysis of natural SOP skills	Instructional SOP (§4.1)	Markdown SKILL.md structure	Empirical structural usage audit	Opaque community skill quality
Gorilla Patil et al. (2024)	Massive API calling with REST docs	Tool & API Calling (§4.2)	REST API call w/ doc constraints	AST sub-tree matching (eval)	API doc hallucination & drift
Toolformer Schick et al. (2023)	Self-supervised tool-calling token insertion	Tool & API Calling (§4.2)	Special API token [API(...) → res]	Self-supervised perplexity filter	Manual tool annotation cost
AnyTool Du et al. (2024)	Hierarchical 16,000+ RapidAPI tool calling	Tool & API Calling (§4.2)	Hierarchical API pool selector	Multi-round API execution feedback	Massive API search space saturation
EasyTool Yuan et al. (2024)	Concise tool instruction wrappers	Tool & API Calling (§4.2)	Concise unified tool schema	None (standardized prompt format)	Excessive API doc token overhead
SkillCraft Chen et al. (2026c)	Benchmark for compositional tool-use skill abstraction	Tool & API Calling (§4.2)	Compositional multi-tool pipelines	Task completion verify + token cost	Instance-level tool usage without reuse
SWE-agent Yang et al. (2024)	Agent-Computer Interfaces for SWE	Software Engineering (§4.4)	ACI custom bash commands	Lint feedback + stdout execution	Unstructured bash format errors
EffiSkill Wang et al. (2026k)	Automated code efficiency optimization skills	Software Engineering (§4.4)	Reusable algorithmic efficiency rules	Runtime execution profiling (speedup)	Correctness vs. efficiency trade-off
SkillDroid Chen et al. (2026b)	Compile once, reuse forever GUI skills	GUI & OS Navigation (§4.4)	Compiled UI transition graph	UI tree state verification	Per-step LLM inference latency
CUA-Skill Chen et al. (2026e)	Reusable computer-using agent skills	GUI & OS Navigation (§4.4)	Hierarchical computer-using abstract	Desktop screenshot + a11y tree diff	Lack of reusable desktop primitives
OS-Expert Liu et al. (2026c)	Professional computer-use agent skills	GUI & OS Navigation (§4.4)	Professional domain OS workflows	End-state eval (OSExpert-Eval)	Amateur vs. professional OS gap
WebVoyager He et al. (2024)	End-to-end multimodal web navigation	GUI & OS Navigation (§4.4)	Multimodal browser clicks & typing	Screenshot ground + DOM observe	Text-only DOM parsing failures
MolmoWeb Ma et al. (2026a)	Open visual web agent with atomic web-skill trajectories	GUI & OS Navigation (§4.4)	Visual browser actions (click/type on screenshot)	Screenshot visual grounding + reward	Proprietary GUI agent opacity & DOM dependence
Odyssey Liu et al. (2025)	Open-world Minecraft skills beyond tech-tree	Embodied & Robotics (§4.4)	Open-world Minecraft skill library	3D voxel world + inventory reward	Tech-tree bias in Minecraft
SELF-VLA Liu et al. (2026a)	Skill-enhanced Vision-Language-Action	Embodied & Robotics (§4.4)	Robotic disassembly motion primitive	Visuotactile force & camera stream	Long-horizon contact manipulation

Notes · Column Value Definitions. **Core Contribution:** Primary domain capability or skill engineering innovation. **Skill Functional Domain:** Primary functional capability or operational environment (*Instructional SOP*: natural-language procedural checklists and guidelines; *Tool & API Calling*: structured schemas for calling external REST APIs or tools; *Software Engineering*: repository-level code editing, refactoring, and debugging; *GUI & OS Navigation*: interacting with digital user interfaces, operating systems, and web browsers; *Embodied & Robotics*: controlling physical robots or 3D embodied game agents). **Action Space Granularity:** Specific primitive action representation operated on by the skill. **Environment Feedback Loop:** Specific feedback signal or observation loop received from the target environment during execution. **Domain Bottleneck Addressed:** Specific operational bottleneck or task complexity challenge solved by skill packaging in this domain.

SOP checklists structured into a three-tier hierarchy: Level 1 Trigger Metadata (30–100 tokens pre-loaded at startup), Level 2 Step-by-Step Instructions (200–5,000 tokens loaded upon activation), and Level 3 Auxiliary Script Resources loaded on-demand.

- **Empirical Audits of Markdown Specifications:** Analyzing community registries in the wild, Ling et al. (2026) (SkillAnalysis) conduct a large-scale structural audit of natural-language SKILL.md packages across major agent platforms. Their findings reveal substantial supply-demand imbalances, high structural redundancy, and significant variation in specification quality across human-authored packages.
- **Concise Schema Compression:** To resolve the context window bloat caused by verbose instructional text, EasyTool (Yuan et al., 2024) demonstrates that pruning extraneous explanatory prose into concise, standardized operational cards substantially reduces instruction token overhead while preserving execution accuracy.

4.1.3 Methodological Trade-offs: Portability vs. Semantic Ambiguity

Instructional SOP skills exhibit maximum cross-model portability: because instructions $\mathcal{I}$ are formulated in natural language, they can be seamlessly loaded by any modern LLM backbone without specialized runtime interpreters. However, this flexibility incurs two severe limitations: (1) *Semantic Ambiguity*, where non-deterministic language interpretations can lead to erratic tool invocations under edge cases; and (2) *Vulnerability to Instruction Injection*, as unverified markdown headers and comments in instructional packages serve as direct vectors for indirect prompt injection (Maloyan & Namiot, 2026; Wang et al., 2026h).

4.2 Tool and API Calling Skills

Tool and API calling skills serve as the operational bridge between the language model’s cognitive planner and external computational environments, translating natural language intent into structured, verifiable system invocations (Schick et al., 2023; Patil et al., 2024).

4.2.1 Formalizing Tool Execution and Parameter Binding

An atomic tool $\tau \in \mathcal{T}$ represents an external function or API endpoint whose execution induces an immediate state update:

$$x_{t+1} = \tau(x_t; \theta_{\text{args}}) \quad (21)$$

where θ_{args} is a vector of parameter bindings validated against a formal JSON Schema or OpenAPI specification. The skill-mediated tool routing policy selects the optimal tool τ^* via:

$$\tau^* = \arg \max_{\tau \in \mathcal{T}} P(\tau \mid x_t, s) \quad (22)$$

under the tool-triggered activation constraint:

$$\mathcal{A}_{\text{tool}}(x_t, g) = 1 \iff \exists \tau \in \mathcal{T} \text{ satisfying sub-goal requirements of } g \quad (23)$$

4.2.2 Comparative Paradigms: From Token Insertion to Compositional Pipelines

The literature approaches tool-augmented capability acquisition across four distinct architectural paradigms:

- **Self-Supervised In-Context Tool Tokenization:** Foundational frameworks such as Toolformer (Schick et al., 2023) integrate tool use directly into the language model’s autoregressive generation. By inserting special API calling tokens $[\text{API}(\text{args}) \rightarrow \text{res}]$ into training sequences, Toolformer optimizes a self-supervised perplexity filter, retaining only those tool invocations that mathematically decrease the cross-entropy loss of subsequent token prediction.
- **Massive API Retrieval and AST Verification:** Addressing massive software libraries ($n > 10^3$ APIs), Gorilla (Patil et al., 2024) fine-tunes models with retrieval-aware instruction tuning over TorchHub, HuggingFace, and TensorHub. To verify functional correctness beyond lexical matching, Gorilla introduces Abstract Syntax Tree (AST) sub-tree matching, eliminating parameter hallucinations.

- **Hierarchical Exploration over Massive Registries:** When candidate API pools scale to tens of thousands ($n = 16,000+$ RapidAPIs), flat bi-encoder retrieval saturates. AnyTool (Du et al., 2024) resolves this saturation by organizing APIs into a hierarchical category tree, utilizing specialized solver and evaluator agents that recursively traverse API sub-trees with self-reflective backtracking.
- **Compositional Tool-Use Skill Abstraction:** Challenging the paradigm of executing isolated, single-turn tools, SkillCraft (Chen et al., 2026c) formalizes *compositional tool skills*, which are reusable macro-routines that chain multiple interdependent APIs into unified execution pipelines. SkillCraft demonstrates that acquiring reusable tool compositions substantially improves out-of-distribution generalization while drastically reducing multi-step token consumption.
- **Concise Schema Normalization:** Raw API documentation frequently contains verbose narrative prose that exhausts prompt context windows. EasyTool (Yuan et al., 2024) addresses this by standardizing diverse API specifications into concise, unified operational cards, substantially eliminating redundant documentation tokens without degrading tool invocation accuracy.

4.2.3 Methodological Trade-offs: Schema Precision vs. Environment Drift

While tool-calling skills provide deterministic system execution and structured return types, they remain vulnerable to *Environment Drift*: when remote REST APIs mutate their endpoints or parameter types, statically cached skill wrappers fail silently unless coupled with automated schema synchronization or runtime contract verifiers (Yuan et al., 2024; Lu et al., 2026b).

4.3 Reasoning and Planning Skills

Unlike tool-calling skills that directly mutate external environment states, reasoning and planning skills operate internally within the agent’s latent cognitive workspace, coordinating structured thought generation, deliberative search, and introspective verification (Sumers et al., 2024; Gou et al., 2024).

4.3.1 Formalizing Latent Thought Traces and Cognitive Scaffolding

Following cognitive agent architectures (Sumers et al., 2024), a reasoning skill r maps the current state $x_t \in \mathcal{X}$ and user goal $g \in \mathcal{G}$ to an explicit sequence of latent thought tokens $z \in \mathcal{Z}$:

$$r : (x_t, g) \rightarrow z = (z_1, z_2, \dots, z_K) \quad (24)$$

where each z_k represents an intermediate cognitive artifact (such as a sub-goal decomposition, hypothesis formulation, mathematical proof step, or counterfactual branch) that conditions the downstream action policy $\pi(a_t | x_t, z)$. The skill activates conditionally via:

$$\mathcal{A}_{\text{reason}}(x_t, g) = 1 \iff \mathcal{H}_{\text{complexity}}(x_t, g) > \delta_{\text{direct}} \quad (25)$$

where $\mathcal{H}_{\text{complexity}}$ evaluates whether the current task requires multi-step deliberative reasoning beyond direct single-step policy evaluation.

4.3.2 Comparative Paradigms: From Introspective Critique to Tool-Interactive Grounding

The literature formalizes reasoning and planning skills across four distinct architectural paradigms:

- **Internal Introspective Refinement:** Foundational frameworks such as Self-Refine (Madaan et al., 2023) encapsulate iterative critique-and-refinement loops within an agent’s internal prompt workspace. By alternating between generation, introspective feedback, and targeted revision without parameter updates, the agent autonomously rectifies syntactic errors and stylistic flaws.
- **Tool-Interactive Verification and Fact Grounding:** Pure internal Chain-of-Thought (CoT) remains prone to hallucination cascades when reasoning over factual or computational domains. CRITIC (Gou et al., 2024) resolves this limitation by making reasoning skills *tool-interactive*: intermediate thought steps z_k are validated against external verifiers (e.g., Python interpreters for numerical calculation, search engines for factual

verification, or static linters for code correctness), creating an empirical feedback loop within the reasoning trace.

- **Deliberate Search and Tree-of-Thought Exploration:** To solve combinatorial and non-linear planning problems, reasoning skills operationalize structured search algorithms, such as Tree-of-Thoughts (Yao et al., 2023a) and Monte Carlo Tree Search (MCTS), where value evaluation heuristics $V(z_k)$ evaluate the promise of intermediate thought states, enabling systematic backtracking out of dead ends.
- **Compositional Skill Synthesis for Complex Reasoning:** In AgenticProposing (Jiao et al., 2026), multi-granularity policy optimization (MGPO) is deployed to synthesize compositional reasoning skills that decompose challenging mathematical proofs, competitive coding, and scientific problem-solving into structured, verifiable checkpoints.
- **Hierarchical Intrinsic Skill Evolution:** Addressing long-horizon decision-making, ARISE (Li et al., 2026e) formalizes reasoning within a bi-level hierarchical reinforcement learning framework, where high-level manager policies select macro-reasoning strategies while intrinsic skill policies execute localized verification loops.

4.3.3 Methodological Trade-offs: Latency Overhead vs. Grounded Verifiability

Reasoning skills substantially enhance problem-solving accuracy on complex reasoning benchmarks; however, they introduce a direct trade-off between *Inference Latency* and *Verification Fidelity*. Multi-step deliberative search scales token consumption proportionally to the branching factor $O(b^d)$, while introspective self-critiquing without external tool grounding risks reinforcing internal hallucinations through confirmation bias (Gou et al., 2024; Madaan et al., 2023).

4.4 Domain-Specific Execution Skills

As autonomous agent deployments expand across specialized computational and physical environments, skill representations must adapt their action-space discretizations, observation feedback loops, and verification bounds to address distinct domain-specific bottlenecks.

4.4.1 Software Engineering (SWE)

Software engineering tasks require agents to navigate complex multi-file codebases, parse abstract syntax trees (ASTs), interact with compilers, and resolve subtle runtime regressions (Jimenez et al., 2024; Han et al., 2026).

- **Agent-Computer Interfaces (ACIs):** Standard bash terminal environments return unconstrained, verbose stdout streams that frequently overflow context windows and trigger formatting syntax errors. To resolve this, SWE-agent (Yang et al., 2024) designs custom Agent-Computer Interfaces (ACIs), which are specialized skill wrappers offering structured file navigation, line-window viewing, and AST-guided edits. ACIs interleave code execution with static linters, immediately trapping syntax errors before git commit generation.
- **Constrained Hierarchical Workflows:** In contrast to unconstrained autonomous loops, Agentless (Xia et al., 2025) demonstrates that decomposing SWE execution into a disciplined two-stage skill pipeline (hierarchical fault localization followed by constrained patch synthesis) matches or outperforms complex autonomous agents while drastically reducing inference cost.
- **Algorithmic Efficiency Optimization:** Moving beyond functional correctness, EffiSkill (Wang et al., 2026k) introduces reusable efficiency-optimization skills on EffiBench-X. By coupling algorithmic refactoring rules with dynamic runtime execution profiling, EffiSkill autonomously refines algorithmic time and memory complexity.

4.4.2 GUI and Operating System Navigation

Computer-using agents operate across desktop operating systems (Windows, macOS, Ubuntu), mobile applications, and web browsers, requiring multi-modal visual grounding over pixel displays and accessibility trees (Xie et al., 2024).

- **Reusable Computer-Use Primitives:** CUA-Skill (Chen et al., 2026e) formalizes reusable computer-using agent skills by coupling structured slot typing with application-specific execution graphs G_e over desktop accessibility (a11y) trees. To master professional multi-step OS workflows, OS-Expert (Liu et al., 2026c) introduces environment-grounded reinforcement learning on OSWorld (Xie et al., 2024), eliminating cascading trajectory errors.
- **Compiled UI State Machines:** Per-step LLM visual inference introduces prohibitive serving latency during repetitive mobile interactions. SkillDroid (Chen et al., 2026b) addresses this by compiling mobile app UI interaction sequences into reusable state transition graphs, achieving high mobile task success while substantially slashing per-step LLM inference calls.
- **Visual Web Grounding:** Addressing DOM tree parsing failures in dynamic web applications, WebVoyager (He et al., 2024) and MolmoWeb (Ma et al., 2026a) ground atomic web-navigation skills directly onto screenshot images. By predicting mouse clicks, scrolling, and keyboard actions conditioned on visual bounding boxes, visual web agents operate robustly without brittle HTML DOM dependencies.

4.4.3 Embodied Agents and Robotics

In physical and simulated embodied environments, skills represent reusable physical control primitives and open-world survival routines operating under continuous kinematics and contact dynamics.

- **Open-World Exploration and Minecraft Survival:** Expanding upon foundational open-ended skill libraries like Voyager (Wang et al., 2024), Odyssey (Liu et al., 2025) constructs open-world survival and exploration skills in Minecraft that transcend rigid tech-tree crafting, evaluating long-term autonomous planning and dynamic immediate exploration.
- **Contact-Rich VLA Manipulation:** In robotic manufacturing and end-of-life electronics disassembly, SELF-VLA (Liu et al., 2026a) integrates Vision-Language-Action (VLA) foundation models with low-level robotic motion primitives, conditioning action policies on real-time visuotactile force feedback and camera streams to ensure safe physical contact.
- **Cross-Embodiment Transfer:** To eliminate the need to relearn physical skills from scratch across diverse robot morphologies, X-Skill (Xu et al., 2023) and UniSkill (Xie et al., 2026a) formalize embodiment-invariant skill discovery, extracting high-level operational abstractions that transfer across heterogeneous robotic arms and kinematic degrees of freedom.

4.4.4 Scientific and Laboratory Discovery

Scientific exploration skills automate hypothesis testing, complex multi-step data pipelines, and symbolic physics reasoning (Mialon et al., 2023).

- **Mining Heterogeneous Scientific Artifacts:** SkillFoundry (Shen et al., 2026) mines scientific code repositories, computational notebooks, and published literature to compile self-evolving skill libraries across MoSciBench and genomics data analysis pipelines, with the majority of mined skills demonstrating novel, reusable scientific analytical capabilities.
- **Symbolic Simulation and Visual Program Synthesis:** Mind’s Eye (Liu et al., 2023) and ScienceWorld (Wang et al., 2022) ground language reasoning in computational physics engines, demonstrating that simulation-augmented reasoning substantially improves scientific problem-solving accuracy over text-only reasoning. Similarly, ViperGPT (Suris et al., 2023) and VisProg (Gupta & Kembhavi, 2023) compose specialized computer vision models into executable Python programmatic skills for spatial scene understanding.

4.5 Collaborative and Meta-Skills

While first-order skills operate directly on external environment states or internal cognitive thoughts, collaborative and meta-skills operate as higher-order operators over the agent’s own capability space $\mathcal{S}$ or coordinate multi-agent societal workflows (Yang et al., 2026b; Zhang et al., 2026b; Li, 2026).

4.5.1 Formalizing Meta-Operators and Second-Order Skill Evolution

We formalize a meta-skill m as a second-order capability transformation that maps an existing skill library $\mathcal{S}$ to an updated, optimized library $\mathcal{S}'$:

$$m : \mathcal{S} \rightarrow \mathcal{S}' \quad (26)$$

The meta-skill activates conditionally when structural deficiencies, semantic redundancy, or interface drift are detected:

$$\mathcal{A}_{\text{meta}}(x_t, g) = 1 \iff \exists s \in \mathcal{S} \text{ requiring diagnostic repair, pruning, or synthesis} \quad (27)$$

Rather than executing single-task actions, meta-skills inspect execution rollouts, diagnose systemic failure modes, and mutate procedural specifications $\mathcal{I}$ or execution policies π to optimize long-term library utility.

4.5.2 Comparative Paradigms: From Self-Curation to Multi-Agent Compilation

The literature approaches meta-skill and collaborative coordination across four distinct paradigms:

- **Autonomous Library Management and Curation:** Treating learned skills as first-class system objects, AutoSkill (Yang et al., 2026b) establishes an experience-driven lifelong learning framework with dedicated skill extraction, consolidation, and pruning layers. Similarly, SkillClaw (Ma et al., 2026c) deploys an open-ended agentic evolver that continually updates community skills without manual user intervention, while Memento (Zhou et al., 2026) consolidates procedural memories across structured subject taxonomies, substantially outperforming uncured baselines across structured subject benchmarks.
- **Co-Evolutionary Verification and Policy Adaptation:** In CoEvoSkills (Zhang et al., 2026b) and Wu et al. (Wu et al., 2026a), the agent decision policy π_{dec} and skill library $\mathcal{M}$ co-evolve simultaneously. High-level planners iteratively refine verification feedback without requiring ground-truth supervision, achieving top pass rates on lifelong benchmarks across both Claude Code and OpenAI Codex backbones.
- **Automated Batch Diagnosis and Optimization:** Addressing silent interface drift, SkillForge (Liu et al., 2026f) implements a three-stage self-optimization pipeline consisting of a *Failure Analyzer*, *Skill Diagnostician*, and *Skill Optimizer*, which diagnoses execution failures in batch, pinpoints underlying procedural deficiencies, and autonomously rewrites defective skill code.
- **Multi-Agent Collaboration vs. Single-Agent Skill Compilation:** In multi-agent frameworks such as MetaGPT (Hong et al., 2024), ChatDev (Qian et al., 2024), and AutoGen (Wu et al., 2024), collaborative skills encode role-specific SOPs and inter-agent communication protocols. Crucially, SingleAgentSkills (Li, 2026) demonstrates that multi-agent SOP collaboration can be compiled into an equivalent single-agent system equipped with modular skill loading, eliminating the vast majority of inter-agent communication token overhead below the critical capability threshold N_{crit} .

4.5.3 Methodological Trade-offs: Self-Repair Stability vs. Cascading Mutation

Meta-skills provide agents with autonomous self-healing and continuous capability growth; however, they introduce the risk of *Second-Order Drift*: if a meta-operator incorrectly modifies core procedural invariants based on noisy or adversarial trajectory feedback, corrupted skill modifications cascade across all downstream tasks that depend on the mutated library (Zhang et al., 2026b; Ma et al., 2026c).

5 Skill Ecosystems and Marketplaces

The rapid proliferation of agent skills has catalyzed a fundamental paradigm shift: transitioning from isolated, single-agent script repositories to open, decentralized, community-driven skill marketplaces and public registries (Li et al., 2026a; Liu et al., 2026i). These ecosystem platforms enable developers to publish, discover, audit, and compose procedural capabilities across diverse agent backbones. A structured comparison of representative evaluation benchmarks, public registries, and ecosystem measurement studies is presented in Table 10.

Table 10: **Ecosystems, Marketplaces & Benchmark Suites (§5)**. Comparison of empirical evaluation benchmarks, skill registries, and marketplace measurement studies ($n = 17$ systems). Sorted primarily by **Ecosystem / benchmark class** (*Lifelong Skill Learning* $\rightarrow$ *Security & Vetting Suite* $\rightarrow$ *Ecosystem Platform / Registry* $\rightarrow$ *Domain Execution Suite*). Demonstrates the transition from static tool-use evaluation to lifelong skill evolution, lifecycle security auditing, and marketplace governance.

System / Suite	Core Contribution	Ecosystem / Benchmark Class	Evaluated Scale / Pool Size	Primary Evaluation Metric	Empirical Finding / Bottleneck
SkillsBench Li et al. (2026d)	First comprehensive agent skill usage benchmark	Lifelong Skill Learning	87 tasks across 8 domains	Pass@1 task completion rate	Skills consistently improve domain pass rates
SkillFlow Zhang et al. (2026e)	20-workflow lifelong skill evolution suite	Lifelong Skill Learning	166 tasks / 20 workflow families	Workflow evolution gain (Δ pass)	Workflow-level skills transfer better than SOPs
SkillLearnBench Zhong et al. (2026)	Continual procedural skill learning benchmark	Lifelong Skill Learning	20 tasks across 15 sub-domains	Skill quality, trajectory & outcome	All CL methods improve over no-skill baseline
SkillSec-Eval Badhe & Tiwari (2026)	Lifecycle-aware security & empirical evaluation	Security & Vetting Suite	327 real-world skills	Attack Success Rate (ASR)	Vulnerabilities span all 5 lifecycle stages
HarmfulSkillBench Jiang et al. (2026b)	Weaponizing agent skills safety benchmark	Security & Vetting Suite	200 harmful skills	ASR & guardrail bypass rate	Substantial fraction harmful; bypasses guards
SkillSafetyBench Jin et al. (2026)	Comprehensive agent skill safety suite	Security & Vetting Suite	155 adversarial cases across 47 tasks	Multi-dimensional ASR score	Natural-language skills create stealthy paths
SkillVetBench Hossain et al. (2026)	LLM-as-a-judge vetting benchmark (SARS score)	Security & Vetting Suite	100 curated skills	Vetting precision / recall / F1	Code-only SAST misses large fraction of markdown exploits
OpenSkillRisk Liu et al. (2026d)	Assessing security risks in open marketplaces	Security & Vetting Suite	263 risky skills across 7 categories	Permission over-request rate	Marketplace skills exhibit high over-privilege
SandboxEscape Marchand et al. (2026)	SANDBOXESCAPEBENCH container escape suite	Security & Vetting Suite	250+ container escape challenges	Sandbox escape success rate	Frontier LLMs achieve significant container escape rate
AgentSkillOS Li et al. (2026a)	OS-like ecosystem for skill orchestration	Ecosystem Platform / Registry	200 to 200,000 skills	Pairwise Bradley-Terry quality score	Tree retrieval + DAG composition beats flat pool
SkillFoundry Shen et al. (2026)	Self-evolving scientific skill repository	Ecosystem Platform / Registry	MoSciBench + genomics	Scientific benchmark pass rate	Large majority of mined skills novel; improves benchmarks
SkillWeaver Zheng et al. (2025)	Weaving skills across OS and web domains	Ecosystem Platform / Registry	200+ web/OS navigation skills	Multi-environment task pass	Hierarchical weaving outperforms flat pools
SkillsInWild Liu et al. (2026i)	Empirical measurement of skills in the wild	In-the-Wild Measurement Study	34,000 real-world skills in the wild	End-to-end task success under noise	Skill utility is fragile; degrades severely without Oracle
OSWorld Xie et al. (2024)	369 multimodal desktop computer-use tasks	Domain Execution Suite	369 multimodal desktop tasks	Desktop task completion rate	Multimodal GUI agents fail on long-horizon OS
TaskBench Shen et al. (2024)	Tool-use & multi-task composition suite	Domain Execution Suite	1,000+ tool-composition tasks	Multi-step DAG success rate	Tool composition fails exponentially w/ depth
BIRD-Bench Wretblad et al. (2024)	Large-scale text-to-SQL w/ external knowledge	Domain Execution Suite	12,751 text-to-SQL tasks	Execution accuracy score	External SQL schema knowledge substantially improves accuracy
OffensiveCS-Eval Chacko et al. (2026)	Negative result on skills in cybersecurity	Domain Execution Suite	120+ cybersecurity CTF tasks	CTF challenge flag capture rate	Procedural skills yield zero gain on CTF tasks

Notes · Column Value Definitions. **Core Contribution:** Primary benchmark, registry, or empirical measurement contribution. **Ecosystem / Benchmark Class:** Primary evaluation scope (*Lifelong Skill Learning*: evaluating skill discovery, evolution, and transfer across multi-session trajectories; *Security & Vetting Suite*: auditing third-party skills, registries, and sandbox containers for vulnerabilities or exploits; *Ecosystem Platform / Registry*: evaluating platforms for managing, caching, and weaving large-scale skill libraries; *Domain Execution Suite*: evaluating agent pass rates across complex desktop, tool-use, database, and cybersecurity tasks). **Evaluated Scale / Pool Size:** Exact reported number of tasks, skills, or repositories evaluated in the empirical experiments. **Primary Evaluation Metric:** Primary mathematical or empirical evaluation signal measured. **Empirical Finding / Bottleneck:** Primary empirical conclusion, scaling behavior, or bottleneck revealed by the benchmark study.

5.1 Public Skill Registries and Marketplaces in the Wild

Modern agent ecosystems increasingly rely on public registries (such as ClawHub, Skills.Rest, and the Claude Code ecosystem) to distribute reusable procedural packages that bundle natural language guidelines $\mathcal{I}$, executable scripts π , and container specifications $\mathcal{E}$ (Li et al., 2026a; Liu et al., 2026i).

5.1.1 Scaling Orchestration to Hundreds of Thousands of Skills

As public registries scale from hundreds to hundreds of thousands of candidate packages ($n > 10^5$), flat bi-encoder vector indexing suffers from severe semantic confusability and ranking saturation. AgentSkillOS (Li et al., 2026a) resolves this bottleneck by organizing massive registries ($n = 200$ to $200,000$ skills) into a hierarchical capability tree coupled with Directed Acyclic Graph (DAG) compilation, demonstrating substantial quality and latency gains over flat pool retrieval. In parallel, SkillWeaver (Zheng et al., 2025) synthesizes cross-domain web and desktop skills into unified executable APIs, achieving substantially higher cross-environment task pass rates over flat pools.

5.1.2 The Empirical In-the-Wild Fragility Gap

While academic evaluations frequently test agents under idealized conditions with hand-curated “oracle” skill subsets, real-world deployment requires autonomous skill discovery from noisy, open pools. In a landmark measurement study across 34,000 real-world skills, Liu et al. (2026i) (SkillsInWild) discover that skill utility in the wild is remarkably fragile: when agents must autonomously retrieve and filter skills from massive community pools, end-to-end task success degrades substantially showing that open retrieval noise substantially degrades end-to-end task completion compared to idealized oracle setups, caused by distractor collisions, vague metadata, and overlapping parameter signatures.

5.2 Lifelong Learning and Continual Evolution Benchmarks

Evaluating an agent’s ability to accumulate, refine, and transfer procedural skills across extended multi-session lifetimes requires standardized continual learning suites:

- **SkillsBench:** As the first comprehensive cross-domain evaluation suite, SkillsBench (Li et al., 2026d) benchmarks 87 complex tasks across 8 operational domains, demonstrating that externalized procedural skills provide consistent pass rate improvements over base LLMs across diverse domains.
- **SkillLearnBench:** SkillLearnBench (Zhong et al., 2026) evaluates continual procedural acquisition across 20 verified tasks in 15 sub-domains, revealing that while continual learning architectures mitigate performance degradation, memory interference and procedural drift remain persistent challenges during extended lifetimes.
- **SkillFlow:** Investigating skill representation granularity, SkillFlow (Zhang et al., 2026e) evaluates lifelong evolution across 20 complex workflow families ($n = 166$ tasks), proving that structured, workflow-level composite skills transfer across structurally analogous tasks significantly better than isolated atomic SOP checklists.

5.3 Security, Vetting, and Governance Ecosystems

The open distribution of unvetted third-party skills introduces critical supply-chain risks, necessitating standardized auditing suites and automated vetting infrastructures:

- **Lifecycle-Wide Threat Quantification:** SkillSec-Eval (Badhe & Tiwari, 2026) establishes a comprehensive threat evaluation framework across 327 enterprise skills in 15 domains, demonstrating that vulnerabilities span all five lifecycle stages, spanning unvetted marketplace admission, Sybil retrieval hijacking, runtime state tampering, and co-evolution poisoning.
- **Behavioral Integrity and Permission Over-Privilege:** A massive audit of 49,943 OpenClaw community skills by Wu et al. (2026b) exposes a widespread description-implementation gap where executable skill routines perform undeclared file access or network requests. Complementarily, OpenSkillRisk (Liu et al.,

2026d) audits 263 risky marketplace skills, revealing widespread over-privilege where benign utilities request root shell and unrestricted directory permissions.

- **Weaponized and Poisoned Registries:** In HarmfulSkillBench (Jiang et al., 2026b), an audit of 98,440 real-world skills reveals that a notable fraction of unvetted registry skills contain weaponized instructions capable of bypassing frontier safety guardrails. Similarly, SkillSafetyBench (Jin et al., 2026) stress-tests 155 adversarial skill injection scenarios across 47 tasks, proving that natural language markdown instructions create stealthy attack pathways that evade standard model alignment.
- **Automated Vetting Leaderboards and Sandbox Isolation:** SkillVetBench (Hossain et al., 2026) establishes an LLM-as-a-judge security vetting benchmark across 100 curated skills using the multi-dimensional Skill Agentic Risk Score (SARS), showing that hybrid semantic judges eliminate false negatives on markdown-layer exploits that bypass traditional SAST scanners. Finally, SandboxEscape (Marchand et al., 2026) demonstrates that frontier LLMs exhibit non-trivial container breakout capabilities when executed in improperly configured Docker sandboxes.

5.4 Domain-Specific Task Execution Environments

Standardized digital testbeds provide the empirical foundation for evaluating skill-augmented agents across complex environments:

- **Desktop and Mobile OS Automation:** OSWorld (Xie et al., 2024) evaluates multimodal agents across 369 open-ended desktop tasks in real Ubuntu, Windows, and macOS installations, while **AndroidInTheWild** (Rawles et al., 2023) benchmarks touch-screen mobile workflows.
- **Web Browsing and Tool Composition Graphs:** WebArena (Zhou et al., 2024) and WebVoyager (He et al., 2024) evaluate multi-page e-commerce and administrative web automation. In multi-tool orchestration, TaskBench (Shen et al., 2024) tests 1,000+ tool composition graphs, proving that execution success drops exponentially as the Directed Acyclic Graph (DAG) dependency depth increases.
- **Software Engineering and Relational Database Querying:** SWE-bench (Jimenez et al., 2024) evaluates end-to-end GitHub issue resolution against unit test suites, while BIRD-Bench (Wretblad et al., 2024) measures text-to-SQL execution accuracy across 12,751 queries, proving that domain-specific database schema skills substantially improve query execution accuracy over raw schema baselines.
- **The CTF Negative Result and Feedback Bandwidth:** In offensive cybersecurity Capture-the-Flag (CTF) challenges across 120+ tasks, OffensiveCS-Eval (Chacko et al., 2026) establishes an essential negative result: static procedural skills provide zero statistically significant performance gain over raw tool baselines. When interactive CLI environments provide immediate terminal stderr tracebacks and exit codes, the environment itself supplies the feedback loop required for self-correction, proving that externalized skill utility is inversely proportional to environmental feedback bandwidth.

6 Open Challenges and Future Directions

Although procedural agent skills have demonstrated significant empirical gains across diverse domains, the field faces foundational theoretical and practical bottlenecks. We synthesize six critical open challenges defining the research frontier:

6.1 Cross-Model Portability and Small Language Model (SLM) Deployment

A fundamental limitation of current procedural skill authoring is its heavy reliance on frontier foundation models (e.g., Claude 3.7, GPT-4.5) to parse complex natural language instructions and resolve ambiguous edge cases. When deployed on Small Language Models (SLMs, 3B–8B parameters) or quantized on-device architectures, skills frequently fail due to context window constraints, strict schema brittleness, and formatting hallucinations (Xu et al., 2026; Ling et al., 2026). Future research must develop *model-agnostic serialization formats* and lightweight execution engines

(e.g., SkillDroid (Chen et al., 2026b), SkillOrchestra (Wang et al., 2026g)) that compile abstract procedural guidelines into compact, verifiable state machines optimized for low-latency edge deployment.

6.2 Sub-Second Dynamic Routing over Massive Registries ($n > 10^5$)

As public registries scale to hundreds of thousands of candidate skills, flat dense embedding retrievers collapse due to semantic crowding, near-synonym confusability, and distractor collisions (Liu et al., 2026i). While two-stage pipelines like SkillRouter (Zheng et al., 2026) demonstrate sub-second median latency at 80K scale via bi-encoder filtering and cross-encoder reranking, scaling to millions of modular functions requires *hierarchical capability trees* and self-reflective meta-routers (AgentSkillOS (Li et al., 2026a)) that dynamically prune search spaces based on evolving task states rather than static initial queries.

6.3 Graph-Structured Compilation and Locality-Bounded Error Recovery

Most contemporary systems execute skills as flat, sequential chains ($s_N \circ \dots \circ s_1$), where any intermediate tool or assertion failure triggers catastrophic trajectory abortion or expensive global replanning ($O(N)$) (Shen et al., 2024). Transitioning to typed Directed Acyclic Graph (DAG) orchestration, as pioneered by GraSP (Xia et al., 2026c), enables *locality-bounded subtree recovery* ($O(d^h)$), repairing only failed dependency nodes. Future architectures must integrate formal pre/post-condition contract invariants (ContractSkill (Lu et al., 2026b)) directly into DAG compilers to enable automatic, provably safe error recovery during runtime execution.

6.4 Zero-Trust Marketplace Security and Formal Program Verification

The open distribution of unvetted community packages creates severe supply-chain vulnerabilities, evidenced by widespread description-implementation gaps across public skills (Wu et al., 2026b) and container breakout risks under misconfigured runtimes (Marchand et al., 2026). Because natural language instructions $\mathcal{I}$ act as porous channels for indirect prompt injection and stealthy exfiltration (Wang et al., 2026h; Schmotz et al., 2025a; Ji et al., 2026), future governance must move toward *Zero-Trust Procedural Architectures*. This requires combining constraint-guided formal Datalog provers (SemiA (Wen et al., 2026)) with hardware-enforced capability sandboxing and cryptographic provenance signatures (SecureAgentSkills (Li et al., 2026f)).

6.5 Lifelong Policy-Skill Co-Evolution without Procedural Drift

In lifelong learning environments where the decision policy π_{dec} and skill memory store $\mathcal{M}$ adapt simultaneously without ground-truth supervision, systems are highly susceptible to *Co-Adaptation Drift*: noisy trajectory rewards reinforce suboptimal heuristics, corrupting downstream skill dependencies (Zhang et al., 2026b; Wu et al., 2026a; Zhong et al., 2026). Resolving this requires developing *virtual procedural paging* (MemGPT (Packer et al., 2023), AutoSkill (Yang et al., 2026b)) and non-parametric semantic policy gradient methods (Skill-Pro (Mi et al., 2026), Skill-R1 (Vishe et al., 2026)) that isolate stable core capabilities while continually refining task-specific parameters.

6.6 The Environmental Feedback Boundary and Lifelong Benchmark Realism

A critical theoretical insight established in our framework is the Environment-Feedback Bandwidth Hypothesis (Chacko et al., 2026): externalized procedural skills provide diminishing marginal utility when interacting with high-bandwidth, schema-validated environments that supply rich diagnostic error traces. Future benchmark suites must move beyond static few-shot evaluation toward interactive, long-horizon testbeds that systematically modulate environmental feedback bandwidth (SkillsBench (Li et al., 2026d), SWE-SkillsBench (Han et al., 2026)). Establishing rigorous empirical boundaries for when skills accelerate convergence versus when they introduce redundant prompt overhead remains a vital open challenge for autonomous agent design.

7 Conclusion

The transition from monolithic prompting and atomic tool use to externalized executable skills represents a critical maturation in the design of autonomous language agents. By decoupling high-level cognitive planning from determinis-

tic, procedural execution, skill-based architectures directly address the context scaling and reliability bottlenecks that plague contemporary LLM systems. This work has formalized the agentic skill abstraction and established a unified lifecycle-oriented reference architecture.

8 Limitations

While this work provides a comprehensive systems foundation for the agentic skills ecosystem, several scope boundaries should be noted. First, the field of autonomous language agents is evolving rapidly; specific runtime frameworks, proprietary APIs, and model-specific tool harnesses will continue to iterate beyond the published systems analyzed here. Second, our primary focus is centered on software engineering, operating system automation, web navigation, and data analysis; while we touch upon embodied robotics and physical simulation, real-world robotic control introduces specialized physical dynamics and hardware latency constraints that warrant dedicated investigation. Finally, our analysis focuses on English-language and Python/JavaScript-centric skill implementations, reflecting the current distribution of open-source registries and public benchmarks.

References

- Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyuan Chen, and Tu Vu. Evoskill: Automated skill discovery for multi-agent systems, 2026. URL <https://arxiv.org/abs/2603.02766>.
- Sanket Badhe and Priyanka Tiwari. Agent skill security: Threat models, attacks, defenses, and evaluation, 2026. URL <https://arxiv.org/abs/2607.13987>.
- Sanket Badhe, Priyanka Tiwari, and Jonghyun Chung. Skill.state: Scalable long-horizon agent skills, 2026. URL <https://arxiv.org/abs/2608.26263>.
- Varun Pratap Bhardwaj. Skillfortify: Formal analysis and supply chain security for agentic ai skills, 2026.
- Shuzhen Bi, Mengsong Wu, Hao Hao, Keqian Li, Wentao Liu, Siyu Song, Hongbo Zhao, and Aimin Zhou. Automating skill acquisition through large-scale mining of open-source agentic repositories: A framework for multi-agent procedural knowledge extraction, 2026. URL <https://arxiv.org/abs/2603.11808>.
- Zouying Cao, Jiaji Deng, Li Yu, Weikang Zhou, Zhaoyang Liu, Bolin Ding, and Hai Zhao. Remember me, refine me: A dynamic procedural memory framework for experience-driven agent evolution. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (eds.), *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 16803–16822, San Diego, California, United States, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-395-1. doi: 10.18653/v1/2026.findings-acl.829. URL <https://aclanthology.org/2026.findings-acl.829/>.
- Samuel Jacob Chacko, James Hugglestone, Chashi Mahiul Islam, and Xiuwen Liu. When skills don’t help: A negative result on procedural knowledge for tool-grounded agents in offensive cybersecurity, 2026. URL <https://arxiv.org/abs/2605.20023>.
- Le Chen, Erhu Feng, Yubin Xia, and Haibo Chen. Skvm: Revisiting language vm for skills across heterogeneous llms and harnesses, 2026a. URL <https://arxiv.org/abs/2604.03088>.
- Qijia Chen, Andrea Bellucci, Zhida Sun, and Giulio Jacucci. Skilldroid: Compile once, reuse forever. *arXiv preprint arXiv:2604.14872*, 2026b.
- Shiqi Chen, Jingze Gai, Ruochen Zhou, Jinghan Zhang, Tongyao Zhu, Junlong Li, Kangrui Wang, Zihan Wang, Zhengyu Chen, Klara Kaleb, et al. Skillcraft: Can llm agents learn to use tools skillfully? *arXiv preprint arXiv:2603.00718*, 2026c.
- Tianhao Chen, Zhengyuan Jiang, Yuepeng Hu, Yebei Gou, and Neil Zhenqiang Gong. Dynamic malicious skills in agentic ai, 2026d. URL <https://arxiv.org/abs/2606.16287>.

-
- Tianyi Chen, Yinheng Li, Michael Solodko, Sen Wang, Nan Jiang, Tingyuan Cui, Junheng Hao, Jongwoo Ko, Sara Abdali, Leon Xu, Suzhen Zheng, Hao Fan, Pashmina Cameron, Justin Wagle, and Kazuhito Koishida. Cua-skill: Develop skills for computer using agent, 2026e. URL <https://arxiv.org/abs/2601.21123>.
- Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=EHg5GDnyq1>.
- Zhihao Chen, Ying Zhang, Yi Liu, Gelei Deng, Yuekang Li, Yanjun Zhang, Jianting Ning, Leo Yu Zhang, Lei Ma, and Zhiqiang Li. Credential leakage in llm agent skills: A large-scale empirical study, 2026f. URL <https://arxiv.org/abs/2604.03070>.
- Hongcheol Cho, Ryangkyung Kang, and Youngeun Kim. Skillret: A large-scale benchmark for skill retrieval in llm agents, 2026. URL <https://arxiv.org/abs/2605.05726>.
- Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. In *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2023. URL <https://openreview.net/forum?id=kiYqb03wqw>.
- Jiawei Du, Jinlong Wu, Yuzheng Chen, Yucheng Hu, Bing Li, and Joey Tianyi Zhou. Rethinking agent design: From top-down workflows to bottom-up skill evolution. *arXiv preprint arXiv:2505.17673*, 2025.
- Yu Du, Fangyun Wei, and Hongyang Zhang. Anytool: self-reflective, hierarchical agents for large-scale api calls. In *Proceedings of the 41st International Conference on Machine Learning, ICML'24*. JMLR.org, 2024.
- Zijian Du and Nathaniel Pinckney. Trace2skill: Verifier-guided skill evolution for long-context eda agents, 2026. URL <https://arxiv.org/abs/2605.21810>.
- Zenghao Duan, Yuxin Tian, Zhiyi Yin, Liang Pang, Jingcheng Deng, Zihao Wei, Shicheng Xu, Yuyao Ge, and Xueqi Cheng. Skillattack: Automated red teaming of agent skills through attack path refinement. *arXiv preprint arXiv:2604.04989*, 2026.
- Bacem Etteib, Daniele Lunghi, and Tégawendé F. Bissyandé. Detecting malicious agent skills in the wild using attention, 2026. URL <https://arxiv.org/abs/2606.23416>.
- Zhiyuan Fan, Tinghao Yu, Yuanjun Cai, Jiangtao Guan, Yun Yang, Dingxin Hu, Jiang Zhou, Xing Wu, Zhuo Han, Feng Zhang, et al. Toward scalable terminal task synthesis via skill graphs. *arXiv preprint arXiv:2604.25727*, 2026.
- Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. Memp: Exploring agent procedural memory, 2026. URL <https://arxiv.org/abs/2508.06433>.
- Yunhao Feng, Yifan Ding, Yingshui Tan, Boren Zheng, Yanming Guo, Xiaolong Li, Kun Zhai, Yishan Li, and Wenke Huang. Skilltrojan: Backdoor attacks on skill-based agent systems. *arXiv preprint arXiv:2604.06811*, 2026.
- Yudong Gao, Zongjie Li, Yuanyuanyuan, Zimo Ji, Pingchuan Ma, and Shuai Wang. Skillreducer: Optimizing llm agent skills for token efficiency, 2026. URL <https://arxiv.org/abs/2603.29919>.
- Jingzhi Gong, Ruizhen Gu, Zhiwei Fei, Yazhuo Cao, Lukas Twist, Alina Geiger, Shuo Han, Dominik Sobania, Federica Sarro, and Jie M. Zhang. Skillmoo: Multi-objective optimization of agent skills for software engineering, 2026. URL <https://arxiv.org/abs/2604.09297>.
- Zhibin Gou, Zhihong Shao, Yeyun Gong, yelong shen, Yujiu Yang, Nan Duan, and Weizhu Chen. CRITIC: Large language models can self-correct with tool-interactive critiquing. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=Sx038qxxjek>.
- Zihan Guo, Zhiyu Chen, Xiaohang Nie, Jianghao Lin, Yuanjian Zhou, and Weinan Zhang. Skillprobe: Security auditing for emerging agent skill marketplaces via multi-agent collaboration. *arXiv preprint arXiv:2603.21019*, 2026.

-
- Tanmay Gupta and Aniruddha Kembhavi. Visual programming: Compositional visual reasoning without training. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu. Swe-skills-bench: Do agent skills actually help in real-world software engineering? *arXiv preprint arXiv:2603.15401*, 2026.
- Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. Webvoyager: Building an end-to-end web agent with large multimodal models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 6864–6890, 2024.
- Florian Holzbauer, David Schmidt, Gabriel Gegenhuber, Sebastian Schrittwieser, and Johanna Ullrich. Malicious or not: Adding repository context to agent skill classification. *arXiv preprint arXiv:2603.16572*, 2026.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VtmBAGCN7o>.
- Ismail Hossain, Sai Puppala, Md Jahangir Alam, Tanzim Ahad, and Sajedul Talukder. Skillvetbench: Llm-as-judge for multi-dimensional security risk evaluation in open-source llm agent skills, 2026. URL <https://arxiv.org/abs/2606.15899>.
- Yinghan Hou and Zongyou Yang. Skillsieve: A hierarchical triage framework for detecting malicious ai agent skills. *arXiv preprint arXiv:2604.06550*, 2026.
- Chia-Yi Hsu, Chia-Mu Yu, Chun-Ying Huang, and Jun Sakuma. Harmless yet harmful: Neutral prompting attacks for stealthy hallucination steering in agent skills, 2026. URL <https://arxiv.org/abs/2605.29354>.
- Haichuan Hu, Ye Shang, and Quanjun Zhang. Red skills or blue skills? a dive into skills published on clawhub. *arXiv preprint arXiv:2604.13064*, 2026a.
- Qisheng Hu, Quanyu Long, and Wenya Wang. When continual learning moves to memory: A study of experience reuse in llm agents, 2026b. URL <https://arxiv.org/abs/2604.27003>.
- Yuepeng Hu, Yuqi Jia, Mengyuan Li, Dawn Song, and Neil Gong. Maltool: Malicious tool attacks on llm agents, 2026c. URL <https://arxiv.org/abs/2602.12194>.
- Xu Huang, Junwu Chen, Yuxing Fei, Zhuohan Li, Philippe Schwaller, and Gerbrand Ceder. Cascade: Cumulative agentic skill creation through autonomous development and evolution, 2026. URL <https://arxiv.org/abs/2512.23880>.
- Zimo Ji, Congying Xu, Zongjie Li, Yudong Gao, Xin Wei, Shuai Wang, and Shing-Chi Cheung. Cloak and detonate: Scanner evasion and dynamic detection of agent skill malware, 2026. URL <https://arxiv.org/abs/2607.02357>.
- Changguo Jia, Tianqi Zhao, Runzhi He, and Minghui Zhou. Skills are not islands: Measuring dependency and risk in agent skill supply chains, 2026a. URL <https://arxiv.org/abs/2607.01136>.
- Xiaojun Jia, Jie Liao, Simeng Qin, Ke Ma, Wenbo Guo, Yebo Feng, Aishan Liu, and Yang Liu. Seeing is not screening: Multimodal hidden instruction attacks on agent skill scanners, 2026b. URL <https://arxiv.org/abs/2606.18198>.
- Guanyu Jiang, Zhaochen Su, Xiaoye Qu, and Yi R Fung. Xskill: Continual learning from experience and skills in multimodal agents. *arXiv preprint arXiv:2603.12056*, 2026a.
- Yukun Jiang, Yage Zhang, Michael Backes, Xinyue Shen, and Yang Zhang. Harmfulskillbench: How do harmful skills weaponize your agents?, 2026b. URL <https://arxiv.org/abs/2604.15415>.
- Zhengbo Jiao, Shaobo Wang, Zifan Zhang, Xuan Ren, Wei Wang, Bing Zhao, Hu Wei, and Linfeng Zhang. Agentic proposing: Enhancing large language model reasoning via compositional skill synthesis, 2026. URL <https://arxiv.org/abs/2602.03279>.

-
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VTF8yNQm66>.
- Chang Jin, An Wang, Zeming Wei, Kai Wang, Biaojie Zeng, Qiaosheng Zhang, Chao Yang, Jingjing Qu, Xia Hu, and Xingcheng Xu. Skillsafetybench: Evaluating agent safety under skill-facing attack surfaces, 2026. URL <https://arxiv.org/abs/2605.12015>.
- Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. An llm compiler for parallel function calling. In *Proceedings of the 41st International Conference on Machine Learning, ICML’24*. JMLR.org, 2024.
- Hao Li, Chunjiang Mu, Jianhao Chen, Siyue Ren, Zhiyao Cui, Yiqun Zhang, Lei Bai, and Shuyue Hu. Organizing, orchestrating, and benchmarking agent skills at ecosystem scale, 2026a. URL <https://arxiv.org/abs/2603.02176>.
- Hao Li et al. Organizing, orchestrating, and benchmarking agent skills at ecosystem scale. *arXiv preprint arXiv:2603.02176*, 2026b. URL <https://arxiv.org/abs/2603.02176>.
- Hongxin Li, Xiping Wang, Jingran Su, Zheng Ju, Yuntao Chen, Qing Li, and Zhaoxiang Zhang. Autogui-v2: A comprehensive multi-modal gui functionality understanding benchmark, 2026c. URL <https://arxiv.org/abs/2604.24441>.
- Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. Api-bank: A comprehensive benchmark for tool-augmented llms. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pp. 3102–3116, 2023.
- Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, Shuyi Wang, Binxu Li, Qunhong Zeng, Di Wang, Xuandong Zhao, Yuanli Wang, Roey Ben Chaim, Zonglin Di, Yipeng Gao, Junwei He, Yizhuo He, Liqiang Jing, Luyang Kong, Xin Lan, Jiachen Li, Songlin Li, Yijiang Li, Yueqian Lin, Xinyi Liu, Xuanqing Liu, Haoran Lyu, Ze Ma, Bowei Wang, Runhui Wang, Tianyu Wang, Wengao Ye, Yue Zhang, Hanwen Xing, Yiqi Xue, Steven Dillmann, and Han chung Lee. Skillsbench: Benchmarking how well agent skills work across diverse tasks, 2026d. URL <https://arxiv.org/abs/2602.12670>.
- Xiaoxiao Li. When single-agent with skills replace multi-agent systems and when they fail. *arXiv preprint arXiv:2601.04748*, 2026.
- Yu Li, Rui Miao, Zhengling Qi, and Tian Lan. Arise: Agent reasoning with intrinsic skill evolution in hierarchical reinforcement learning, 2026e. URL <https://arxiv.org/abs/2603.16060>.
- Yuyang Li, Yiran Dou, Jie-Jing Shao, Yueming Lyu, Ivor Tsang, and Haiyan Yin. Skilltracer: Structural failure attribution and refinement of agentic skills in long-horizon web tasks. In *Workshop on Multi-Agent Learning and Its Opportunities in the Era of Generative AI*.
- Zhiyuan Li, Jingzheng Wu, Xiang Ling, Xing Cui, and Tianyue Luo. Towards secure agent skills: Architecture, threat taxonomy, and security analysis. *arXiv preprint arXiv:2604.02837*, 2026f.
- Zhiyuan Li, Jingzheng Wu, Xiang Ling, Xing Cui, and Tianyue Luo. Towards secure agent skills: Architecture, threat taxonomy, and security analysis, 2026g. URL <https://arxiv.org/abs/2604.02837>.
- Yuan Liang, Ruobin Zhong, Haoming Xu, Chen Jiang, Yi Zhong, Runnan Fang, Jia-Chen Gu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, Xin Xu, Tongtong Wu, Kun Wang, Yang Liu, Zhen Bi, Jungang Lou, Yuchen Eleanor Jiang, Hangcheng Zhu, Gang Yu, Haiwen Hong, Longtao Huang, Hui Xue, Chenxi Wang, Yijun Wang, Zifei Shan, Xi Chen, Zhaopeng Tu, Feiyu Xiong, Xin Xie, Peng Zhang, Zhengke Gui, Lei Liang, Jun Zhou, Chiyu Wu, Jin Shang, Yu Gong, Junyu Lin, Changliang Xu, Hongjie Deng, Wen Zhang, Keyan Ding, Qiang Zhang, Fei Huang, Ningyu Zhang, Jeff Z. Pan, Guilin Qi, Haofen Wang, and Huajun Chen. Skillnet: Create, evaluate, and connect ai skills, 2026. URL <https://arxiv.org/abs/2603.04448>.

-
- Yu-Ting Lin and Chia-Mu Yu. Phantomskill: Malicious code injection in agent skill ecosystems, 2026. URL <https://arxiv.org/abs/2606.19191>.
- George Ling, Shanshan Zhong, and Richard Huang. Agent skills: A data-driven analysis of claude skills for extending large language model functionality. *arXiv preprint arXiv:2602.08004*, 2026.
- Anthony Zhe Liu, Jongwook Choi, Sungryull Sohn, Yao Fu, Jaekyeom Kim, Dong-Ki Kim, Xinhe Wang, Jaewon Yoo, and Honglak Lee. Skillact: Using skill abstractions improves llm agents. In *ICML 2024 Workshop on LLMs and Cognition*, 2024.
- Chang Liu, Sibotian, Xiao Liang, and Minghui Zheng. Self-vla: A skill enhanced agentic vision-language-action framework for contact-rich disassembly, 2026a. URL <https://arxiv.org/abs/2603.11080>.
- Dawei Liu, Zongxia Li, Hongyang Du, Xiyang Wu, Shihang Gui, Yongbei Kuang, and Lichao Sun. Graph of skills: Dependency-aware structural retrieval for massive agent skills, 2026b. URL <https://arxiv.org/abs/2604.05333>.
- Jiateng Liu, Zhenhailong Wang, Rushi Wang, Bingxuan Li, Jeonghwan Kim, Aditi Tiwari, Pengfei Yu, Denghui Zhang, and Heng Ji. Osexpert: Computer-use agents learning professional skills via exploration. *arXiv preprint arXiv:2603.07978*, 2026c.
- Qiyuan Liu, Tingfeng Hui, Kun Zhan, Kaike Zhang, and Ning Miao. Openskillrisk: Benchmarking agent safety when using real-world risky third-party skills, 2026d. URL <https://arxiv.org/abs/2607.20121>.
- Ruibo Liu, Jason Wei, Shixiang Shane Gu, Te-Yen Wu, Soroush Vosoughi, Claire Cui, Denny Zhou, and Andrew M. Dai. Mind’s eye: Grounded language model reasoning through simulation. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=4rXMRuoJlai>.
- Shunyu Liu, Yaoru Li, Kongcheng Zhang, Zhenyu Cui, Wenkai Fang, Yuxuan Zheng, Tongya Zheng, and Mingli Song. Odyssey: Empowering minecraft agents with open-world skills, 2025. URL <https://arxiv.org/abs/2407.15325>.
- Xingyan Liu, Xiyue Luo, Linyu Li, Ganghong Huang, Jianfeng Liu, and Honglin Qiao. Skillforge: Forging domain-specific, self-evolving agent skills in cloud technical support. *arXiv preprint arXiv:2604.08618*, 2026e.
- Xingyan Liu, Xiyue Luo, Linyu Li, Ganghong Huang, Jianfeng Liu, and Honglin Qiao. Skillforge: Forging domain-specific, self-evolving agent skills in cloud technical support. In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 4763–4768. ACM, 2026f. doi: 10.1145/3805712.3808466. URL <http://dx.doi.org/10.1145/3805712.3808466>.
- Yi Liu, Zhihao Chen, Yanjun Zhang, Gelei Deng, Yuekang Li, Jianting Ning, Ying Zhang, and Leo Yu Zhang. Malicious agent skills in the wild: A large-scale security empirical study, 2026g. URL <https://arxiv.org/abs/2602.06547>.
- Yi Liu, Weizhe Wang, Ruitao Feng, Yao Zhang, Guangquan Xu, Gelei Deng, Yuekang Li, and Leo Zhang. Agent skills in the wild: An empirical study of security vulnerabilities at scale, 2026h. URL <https://arxiv.org/abs/2601.10338>.
- Yujian Liu, Jiabao Ji, Li An, Tommi Jaakkola, Yang Zhang, and Shiyu Chang. How well do agentic skills work in the wild: Benchmarking llm skill usage in realistic settings, 2026i. URL <https://arxiv.org/abs/2604.04323>.
- Zhengxi Lu, Zhiyuan Yao, Jinyang Wu, Chengcheng Han, Qi Gu, Xunliang Cai, Weiming Lu, Jun Xiao, Yueting Zhuang, and Yongliang Shen. Skill0: In-context agentic reinforcement learning for skill internalization. *arXiv preprint arXiv:2604.02268*, 2026a.
- Zijian Lu, Yiping Zuo, Yupeng Nie, Xin He, Weibei Fan, Lianyong Qi, and Shi Jin. Contractskill: Repairable contract-based skills for multimodal web agents, 2026b. URL <https://arxiv.org/abs/2603.20340>.
- Junyu Luo, Weizhi Zhang, Ye Yuan, Yusheng Zhao, Junwei Yang, Yiyang Gu, Bohan Wu, Binqi Chen, Ziyue Qiao, Qingqing Long, Rongcheng Tu, Xiao Luo, Wei Ju, Zhiping Xiao, Yifan Wang, Meng Xiao, Chenwu Liu, Jingyang Yuan, Shichang Zhang, Yiqiao Jin, Fan Zhang, Xian Wu, Hanqing Zhao, Dacheng Tao, Philip S. Yu, and Ming Zhang. Large language model agent: A survey on methodology, applications and challenges, 2025. URL <https://arxiv.org/abs/2503.21460>.

-
- Lijia Lv, Xuehai Tang, Jie Wen, Jizhong Han, and Songlin Hu. Structured security auditing and robustness enhancement for untrusted agent skills. *arXiv preprint arXiv:2604.25109*, 2026.
- Yingwei Ma, Yue Liu, Xinlong Yang, Yanhao Li, Kelin Fu, Yibo Miao, Yuchong Xie, Zhexu Wang, and Shing-Chi Cheung. Scaling coding agents via atomic skills. *arXiv preprint arXiv:2604.05013*, 2026a.
- Yuchen Ma, Yue Huang, Han Bao, Haomin Zhuang, Swadheen Shukla, Michel Galley, Xiangliang Zhang, and Stefan Feuerriegel. Skillgen: Verified inference-time agent skill synthesis, 2026b. URL <https://arxiv.org/abs/2605.10999>.
- Ziyu Ma, Shidong Yang, Yuxiang Ji, Xucong Wang, Yong Wang, Yiming Hu, Tongwen Huang, and Xiangxiang Chu. Skillclaw: Let skills evolve collectively with agentic evolver. *arXiv preprint arXiv:2604.08377*, 2026c.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhunoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback, 2023. URL <https://arxiv.org/abs/2303.17651>.
- Narek Maloyan and Dmitry Namiot. Prompt injection attacks on agentic coding assistants: A systematic analysis of vulnerabilities in skills, tools, and protocol ecosystems. *International Journal of Open Information Technologies*, 14(2):1–10, 2026.
- Rahul Marchand, Art O Cathain, Jerome Wynne, Philippos Maximos Giavridis, Sam Deverett, John Wilkinson, Jason Gwartz, and Harry Coppock. Quantifying frontier llm capabilities for container sandbox escape, 2026. URL <https://arxiv.org/abs/2603.02277>.
- Qirui Mi, Zhijian Ma, Mengyue Yang, Haoxuan Li, Yisen Wang, Haifeng Zhang, and Jun Wang. Skill-pro: Learning reusable skills from experience via non-parametric ppo for llm agents. *arXiv preprint arXiv:2602.01869*, 2026.
- Grégoire Mialon, Roberto Dessì, Maria Lomeli, Christoforos Nalmpantis, Ram Pasunuru, Roberta Raileanu, Baptiste Rozière, Timo Schick, Jane Dwivedi-Yu, Asli Celikyilmaz, Edouard Grave, Yann LeCun, and Thomas Scialom. Augmented language models: a survey, 2023. URL <https://arxiv.org/abs/2302.07842>.
- Microsoft. Prompt flow: Build high-quality llm apps from prototyping to production. 2024.
- Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Erchao Zhao, Xiaoxi Jiang, and Guanjin Jiang. Trace2skill: Distill trajectory-local lessons into transferable agent skills. *arXiv preprint arXiv:2603.25158*, 2026.
- Zheng Nie, Ruolin Shen, Xinlei Yu, Bo Yin, Jiangning Zhang, and Xiaobin Hu. Skillgraph: Self-evolving multi-agent collaboration with multimodal graph topology. *arXiv preprint arXiv:2604.17503*, 2026.
- Siru Ouyang, Jun Yan, Yanfei Chen, Rujun Han, Zifeng Wang, Bhavana Dalvi Mishra, Rui Meng, Chun-Liang Li, Yizhu Jiao, Kaiwen Zha, et al. Skillos: Learning skill curation for self-evolving agents. *arXiv preprint arXiv:2605.06614*, 2026.
- Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. *CoRR*, abs/2310.08560, 2023. URL <https://doi.org/10.48550/arXiv.2310.08560>.
- Joon Sung Park, Joseph C O’Brien, Carrie J Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, pp. 1–22, 2023.
- Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=tBRNC6YemY>.
- Hongji Pu, Xinyuan Song, and Liang Zhao. Skillops: Managing llm agent skill libraries as self-maintaining software ecosystems, 2026. URL <https://arxiv.org/abs/2605.13716>.

-
- Denys Pushkin and Emmanuel Abbe. Lead: Breaking the no-recovery bottleneck in long-horizon reasoning, 2026. URL <https://arxiv.org/abs/2603.06870>.
- Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, et al. Chatdev: Communicative agents for software development. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)*, pp. 15174–15186, 2024.
- Libin Qiu, Zhirong Gao, Junfu Chen, Yuhang Ye, Weizhi Huang, Xiaobo Xue, Wenkai Qiu, and Shuo Tang. Autorefine: From trajectories to reusable expertise for continual llm agent refinement, 2026. URL <https://arxiv.org/abs/2601.22758>.
- Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy P Lillicrap. Androidinthewild: A large-scale dataset for android device control. In *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2023. URL <https://openreview.net/forum?id=j4b315k0i1>.
- Sampriti Saha and Pranav Hemanth. Skilldex: A package manager and registry for agent skill packages with hierarchical scope-based distribution. *arXiv preprint arXiv:2604.16911*, 2026.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=Yacmpz84TH>.
- David Schmotz, Sahar Abdelnabi, and Maksym Andriushchenko. Agent skills enable a new class of realistic and trivially simple prompt injections, 2025a. URL <https://arxiv.org/abs/2510.26328>.
- David Schmotz, Sahar Abdelnabi, and Maksym Andriushchenko. Agent skills enable a new class of realistic and trivially simple prompt injections. *arXiv preprint arXiv:2510.26328*, 2025b.
- Shuaike Shen, Wenduo Cheng, Mingqian Ma, Alistair Turcan, Martin JinYE Zhang, and Jian Ma. Skillfoundry: Building self-evolving agent skill libraries from heterogeneous scientific resources. *arXiv preprint arXiv:2604.03964*, 2026.
- Yongliang Shen, Kaitao Song, Xu Tan, Wenqi Zhang, Kan Ren, Siyu Yuan, Weiming Lu, Dongsheng Li, and Yueting Zhuang. Taskbench: Benchmarking large language models for task automation. *Advances in Neural Information Processing Systems*, 37:4540–4574, 2024.
- Haochen Shi, Xingdi Yuan, and Bang Liu. Evolving programmatic skill networks, 2026. URL <https://arxiv.org/abs/2601.03509>.
- Tianmin Shu, Caiming Xiong, and Richard Socher. Hierarchical and interpretable skill acquisition in multi-task reinforcement learning, 2017. URL <https://arxiv.org/abs/1712.07294>.
- Weihsang Su, Jianming Long, Qingyao Ai, Yichen Tang, Changyue Wang, Yiteng Tu, and Yiqun Liu. Skill retrieval augmentation for agentic ai. *arXiv preprint arXiv:2604.24594*, 2026.
- Theodore Sumers, Shunyu Yao, Karthik R Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL <https://openreview.net/forum?id=1i6ZCvf1QJ>. Survey Certification, Featured Certification.
- Yiqun Sun, Pengfei Wei, and Lawrence B. Hsieh. Don’t retrieve, navigate: Distilling enterprise knowledge into navigable agent skills for qa and rag, 2026. URL <https://arxiv.org/abs/2604.14572>.
- Didac Suris, Ruoshi Gupta, and Carl Vondrick. Vipergpt: Visual grounding via python execution. In *IEEE International Conference on Computer Vision (ICCV)*, 2023.
- Guiyao Tie, Jiawen Shi, Pan Zhou, and Lichao Sun. Badskill: Backdoor attacks on agent skills via model-in-skill poisoning, 2026. URL <https://arxiv.org/abs/2604.09378>.
- Songjun Tu, Chengdong Xu, Qichao Zhang, Yaocheng Zhang, Xiangyuan Lan, Linjing Li, Dong Li, and Dongbin Zhao. Dynamic dual-granularity skill bank for agentic rl, 2026. URL <https://arxiv.org/abs/2603.28716>.

-
- Georgios Tzifas and Hamidreza Kasaei. Lifelong robot library learning: Bootstrapping composable and generalizable skills for embodied control with language models. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 515–522. IEEE, 2024.
- Yash Vishe, Rohan Surana, Xunyi Jiang, Zihan Huang, Xintong Li, Nikki Lijing Kuang, Tong Yu, Ryan A. Rossi, Jingbo Shang, Julian McAuley, and Junda Wu. Skill-r1: Agent skill evolution via reinforcement learning, 2026. URL <https://arxiv.org/abs/2605.09359>.
- Chenxi Wang, Zhuoyun Yu, Xin Xie, Wuguannan Yao, Runnan Fang, Shuofei Qiao, Kexin Cao, Guozhou Zheng, Xiang Qi, Peng Zhang, et al. Skillx: Automatically constructing skill knowledge bases for agents. *arXiv preprint arXiv:2604.04804*, 2026a.
- Fali Wang, Chenglin Weng, Xianren Zhang, Siyuan Hong, Hui Liu, and Suhang Wang. Graphskill: Documentation-guided hierarchical retrieval-augmented coding for complex graph reasoning, 2026b. URL <https://arxiv.org/abs/2603.06620>.
- Fali Wang, Chenglin Weng, Xianren Zhang, Siyuan Hong, Hui Liu, and Suhang Wang. Graphskill: Documentation-guided hierarchical retrieval-augmented coding for complex graph reasoning, 2026c. URL <https://arxiv.org/abs/2603.06620>.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL <https://openreview.net/forum?id=ehfRiF0R3a>.
- Hanyu Wang, Yifan Lan, Bochuan Cao, Lu Lin, and Jinghui Chen. Skillgrad: Optimizing agent skills like gradient descent, 2026d. URL <https://arxiv.org/abs/2605.27760>.
- Hao Wang, Guozhi Wang, Han Xiao, Yufeng Zhou, Yue Pan, Jichao Wang, Ke Xu, Yafei Wen, Xiaohu Ruan, and Xiaoxin Chen. Skill-conditioned self-distillation for multi-turn llm agents. *arXiv preprint arXiv:2604.10674*, 2026e.
- Jiayu Wang, Yifei Ming, Zixuan Ke, Shafiq Joty, Aws Albarghouthi, and Frederic Sala. Skillorchestra: Learning to route agents via skill transfer, 2026f. URL <https://arxiv.org/abs/2602.19672>.
- Jiayu Wang, Yifei Ming, Zixuan Ke, Shafiq Joty, Aws Albarghouthi, and Frederic Sala. Skillorchestra: Learning to route agents via skill transfer. *arXiv preprint arXiv:2602.19672*, 2026g.
- Jiong Xiao Wang, Qiaojing Yan, Yawei Wang, Yijun Tian, Soumya Smruti Mishra, Zhichao Xu, Megha Gandhi, Panpan Xu, and Lin Lee Cheong. Reinforcement learning for self-improving agent with skill library. *arXiv preprint arXiv:2512.17102*, 2025a.
- Qianli Wang, Boyang Ma, Minghui Xu, and Yue Zhang. When skills lie: Hidden-comment injection in llm agents. *arXiv preprint arXiv:2602.10498*, 2026h.
- Ruoyao Wang, Peter Jansen, Marc-Alexandre Côté, and Prithviraj Ammanabrolu. ScienceWorld: Is your agent smarter than a 5th grader? In Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (eds.), *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 11279–11298, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.emnlp-main.775. URL <https://aclanthology.org/2022.emnlp-main.775/>.
- Xinyu Jessica Wang, Haoyue Bai, Yiyi Sun, Haorui Wang, Shuibai Zhang, Wenjie Hu, Mya Schroder, Bilge Mutlu, Dawn Song, and Robert D Nowak. The long-horizon task mirage? diagnosing where and why agentic systems break, 2026i. URL <https://arxiv.org/abs/2604.11978>.
- Zhaoyang Wang, Qianhui Wu, Xuchao Zhang, Chaoyun Zhang, Wenlin Yao, Fazle Elahi Faisal, Baolin Peng, Si Qin, Suman Nath, Qingwei Lin, et al. Webxskill: Skill learning for autonomous web agents. *arXiv preprint arXiv:2604.13318*, 2026j.
- Zimu Wang, Yuling Shi, Mengfan Li, Zijun Liu, Jie M. Zhang, Chengcheng Wan, and Xiaodong Gu. Effiskill: Agent skill based automated code efficiency optimization, 2026k. URL <https://arxiv.org/abs/2603.27850>.

-
- Zora Zhiruo Wang, Apurva Gandhi, Graham Neubig, and Daniel Fried. Inducing programmatic skills for agentic tasks. *arXiv preprint arXiv:2504.06821*, 2025b.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Hongbo Wen, Ying Li, Hanzhi Liu, Chaofan Shou, Yanju Chen, Yuan Tian, and Yu Feng. Semia: Auditing agent skills via constraint-guided representation synthesis, 2026. URL <https://arxiv.org/abs/2605.00314>.
- Zikai Alex Wen. Toward user comprehension supports for LLM agent skill specifications. In *First Workshop on Agent Skills*, 2026. URL <https://openreview.net/forum?id=RYDIwFbaBd>.
- Niklas Wretblad, Fredrik Gordh Riseby, Rahul Biswas, Amin Ahmadi, and Oskar Holmström. Understanding the effects of noise in text-to-sql: An examination of the bird-bench benchmark, 2024. URL <https://arxiv.org/abs/2402.12243>.
- Haolin Wu, Yuecheng Liu, Junyi Dong, Heng Zhang, Sitong Mao, Hesheng Wang, Weigang Wu, and Shunbo Zhou. Ascent: Autonomous skill learning toward complex embodied tasks with foundation models. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 16752–16758, 2025. doi: 10.1109/ICRA55743.2025.11127927.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen LLM applications via multi-agent conversations. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=BAakY1hNKS>.
- Xiyang Wu, Zongxia Li, Guangyao Shi, Alexander Duffy, Tyler Marques, Matthew Lyle Olson, Tianyi Zhou, and Dinesh Manocha. Co-evolving llm decision and skill bank agents for long-horizon tasks, 2026a. URL <https://arxiv.org/abs/2604.20987>.
- Yuhao Wu, Tung-Ling Li, and Hongliang Liu. Behavioral integrity verification for ai agent skills, 2026b. URL <https://arxiv.org/abs/2605.11770>.
- Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Demystifying llm-based software engineering agents. *Proc. ACM Softw. Eng.*, 2(FSE), June 2025. doi: 10.1145/3715754. URL <https://doi.org/10.1145/3715754>.
- Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, et al. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning. *arXiv preprint arXiv:2602.08234*, 2026a.
- Tianle Xia, Lingxiang Hu, Yiding Sun, Ming Xu, Lan Xu, Siying Wang, Wei Xu, and Jie Jiang. Grasp: Graph-structured skill compositions for llm agents. *arXiv preprint arXiv:2604.17870*, 2026b. URL <https://arxiv.org/abs/2604.17870>.
- Tianle Xia, Lingxiang Hu, Yiding Sun, Ming Xu, Lan Xu, Siying Wang, Wei Xu, and Jie Jiang. Grasp: Graph-structured skill compositions for llm agents, 2026c. URL <https://arxiv.org/abs/2604.17870>.
- Wenjie Xiao, Xuehai Tang, Biyu Zhou, Songlin Hu, and Jizhong Han. Routeguard: Internal-signal detection of skill poisoning in llm agents. *arXiv preprint arXiv:2604.22888*, 2026.
- Senwei Xie, Yuntian Zhang, Ruiping Wang, and Xilin Chen. Uni-skill: Building self-evolving skill repository for generalizable robotic manipulation, 2026a. URL <https://arxiv.org/abs/2603.02623>.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments, 2024. URL <https://arxiv.org/abs/2404.07972>.

-
- Yi Xie, Jiawei Du, Yu Cheng, Jiuan Zhou, and Zhaoxia Yin. Benign in isolation, harmful in composition: Security risks in agent skill ecosystems, 2026b. URL <https://arxiv.org/abs/2606.15242>.
- Yuquan Xie, Zaijing Li, Rui Shao, Gongwei Chen, Kaiwen Zhou, Yinchuan Li, Dongmei Jiang, and Liqiang Nie. Mirage-1: Augmenting and updating gui agent with hierarchical multimodal skills. *arXiv preprint arXiv:2506.10387*, 2025.
- Hanwen Xing, Haomin Zhuang, Xuandong Zhao, Yue Huang, Zhenheng Tang, and Xiangliang Zhang. Recipes for agents: Understanding skills and their open questions. *Preprint, ResearchGate*. doi, 10, 2026.
- Mengda Xu, Zhenjia Xu, Cheng Chi, Manuela Veloso, and Shuran Song. Xskill: Cross embodiment skill discovery, 2023. URL <https://arxiv.org/abs/2307.09955>.
- Yangjie Xu, Lujun Li, Lama Sleem, Niccolo Gentile, Yewei Song, Yiqun Wang, Siming Ji, Wenbo Wu, and Radu State. Agent skill framework: Perspectives on the potential of small language models in industrial environments, 2026. URL <https://arxiv.org/abs/2602.16653>.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering, 2024. URL <https://arxiv.org/abs/2405.15793>.
- Yifan Yang, Ziyang Gong, Weiwan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Yuqing Yang, Dongdong Chen, Xue Yang, and Chong Luo. Skillopt: Executive strategy for self-evolving agent skills, 2026a. URL <https://arxiv.org/abs/2605.23904>.
- Yongjin Yang, Sinjae Kang, Juyong Lee, Dongjun Lee, Se-Young Yun, and Kimin Lee. Automated skill discovery for language agents through exploration and iterative feedback, 2025a. URL <https://arxiv.org/abs/2506.04287>.
- Yutao Yang, Junsong Li, Qianjun Pan, Bihao Zhan, Yuxuan Cai, Lin Du, Jie Zhou, Kai Chen, Qin Chen, Xin Li, Bo Zhang, and Liang He. Autoskill: Experience-driven lifelong learning via skill self-evolution, 2026b. URL <https://arxiv.org/abs/2603.01145>.
- Zonghan Yang, Shengjie Wang, Kelin Fu, Wenyang He, Weimin Xiong, Yibo Liu, Yibo Miao, Bofei Gao, Yejie Wang, Yingwei Ma, et al. Kimi-dev: Agentless training as skill prior for swe-agents. *arXiv preprint arXiv:2509.23045*, 2025b.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models, 2023a. URL <https://arxiv.org/abs/2305.10601>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023b.
- Haoran Ye, Xuning He, Vincent Arak, Haonan Dong, and Guojie Song. Meta context engineering via agentic skill evolution. *arXiv preprint arXiv:2601.21557*, 2026.
- Simon Yu, Gang Li, Weiyan Shi, and Peng Qi. Polyskill: Learning generalizable skills through polymorphic abstraction. *arXiv preprint arXiv:2510.15863*, 2025.
- Haoqi Yuan, Chi Zhang, Hongcheng Wang, Feiyang Xie, Penglin Cai, Hao Dong, and Zongqing Lu. Skill reinforcement learning and planning for open-world long-horizon tasks. *arXiv preprint arXiv:2303.16563*, 2023.
- Siyu Yuan, Kaitao Song, Jiangjie Chen, Xu Tan, Yongliang Shen, Ren Kan, Dongsheng Li, and Deqing Yang. Easytool: Enhancing llm-based agents with concise tool instruction, 2024. URL <https://arxiv.org/abs/2401.06201>.
- Renos Zabounidis, Yue Wu, Simon Stepputtis, Woojun Kim, Yanzhi Li, Tom Mitchell, and Katia Sycara. Scalar: Learning and composing skills through llm guided symbolic planning and deep rl grounding, 2026. URL <https://arxiv.org/abs/2603.09036>.

-
- Guijia Zhang, Shu Yang, Xilin Gong, and Di Wang. Stars: Skill-triggered audit for request-conditioned invocation safety in agent systems. *arXiv preprint arXiv:2604.10286*, 2026a.
- Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, Xue Liu, Xiaoxiao Li, and Philip S. Yu. Coevoskills: Self-evolving agent skills via co-evolutionary verification, 2026b. URL <https://arxiv.org/abs/2604.01687>.
- Haozhen Zhang, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, and Wenya Wang. Memskill: Learning and evolving memory skills for self-evolving agents. *arXiv preprint arXiv:2602.02474*, 2026c.
- Xing Zhang, Guanghui Wang, Yanwei Cui, Wei Qiu, Ziyuan Li, Bing Zhu, and Peiyang He. Experience compression spectrum: Unifying memory, skills, and rules in llm agents, 2026d. URL <https://arxiv.org/abs/2604.15877>.
- Ziao Zhang, Kou Shi, Shiting Huang, Avery Nie, Yu Zeng, Yiming Zhao, Zhen Fang, Qishen Su, Haibo Qiu, Wei Yang, et al. Skillflow: Benchmarking lifelong skill discovery and evolution for autonomous agents. *arXiv preprint arXiv:2604.17308*, 2026e.
- Boyuan Zheng, Michael Y Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, et al. Skillweaver: Web agents can self-improve by discovering and honing skills. *arXiv preprint arXiv:2504.07079*, 2025.
- YanZhao Zheng, ZhenTao Zhang, Chao Ma, YuanQiang Yu, JiHuai Zhu, Yong Wu, Tianze Xu, Baohua Dong, Hangcheng Zhu, Ruohui Huang, and Gang Yu. Skillrouter: Skill routing for llm agents at scale, 2026. URL <https://arxiv.org/abs/2603.22455>.
- Shanshan Zhong, Yi Lu, Jingjie Ning, Yibing Wan, Lihan Feng, Yuyi Ao, Leonardo FR Ribeiro, Markus Dreyer, Sean Ammirati, and Chenyan Xiong. Skilllearnbench: Benchmarking continual learning methods for agent skill generation on real-world tasks. *arXiv preprint arXiv:2604.20087*, 2026.
- Huichi Zhou, Siyuan Guo, Anjie Liu, Zhongwei Yu, Ziqin Gong, Bowen Zhao, Zhixun Chen, Menglong Zhang, Yihang Chen, Jinsong Li, et al. Memento-skills: Let agents design agents. *arXiv preprint arXiv:2603.18743*, 2026.
- Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents, 2024. URL <https://arxiv.org/abs/2307.13854>.
- Zhaojiacheng Zhou. Proteus: A self-evolving red team for agent skill ecosystems, 2026. URL <https://arxiv.org/abs/2605.11891>.
- Jiaying Zhu, Lyuye Zhang, Wenbo Guo, and Yang Liu. Skillclone: Multi-modal clone detection and clone propagation analysis in the agent skill ecosystem. *arXiv preprint arXiv:2603.22447*, 2026.
- Haomin Zhuang, Hanwen Xing, and Xiangliang Zhang. Agentclick: A skill-based human-in-the-loop review layer for terminal ai agents, 2026. URL <https://arxiv.org/abs/2604.16520>.